VistaPoint

Version 4.0

Reference Manual

(The XnslHTML Widget Class)

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:

Nova Software Labs
8, rue de Hanovre
75002 Paris, France
Tel: +33 1 58 18 61 61
Fax: +33 1 58 18 61 60
email: sales@NovaSoftwareLabs.com or sales@nsl.fr.
Web: http://www.NovaSoftwareLabs.com or http://www.nsl.fr.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker is a registered trademark of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: VistaPoint v. 7.1.1.
Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of XFaceMaker:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Table of Contents

CHAPTER 1: Introduction

This manual describes the VistaPoint widget. This widget is designed to read, interpret and display HTML (Hyper Text Markup Language) version 3.2 text and images.

VistaPoint is fully compliant with correct HTML 3.2 source, as described in the following document:

> *HTML 3.2 Reference Specification,* the W3C Recommendation 14-Jan-1997, by Dave Raggett. This document is available on the World Wide Web, at: `http://www.w3.org/TR/REC-html32`

If it is fed correct input code, the widget will render the page according to the specifications.

If it is fed incorrect code, the widget will do its best to handle it. It is able to handle many of the syntax errors that are common in the HTML pages on the World Wide Web. VistaPoint detects missing closures and inserts missing tags where needed.

For example, if the widget were fed the following faulty HTML code:

```
<table border=1 cellpadding=0 cellspacing=0 width=100%>
    some text
    <caption>
        a caption
    <tr valign=middle>
        <td>
            <strong> some more text
        <td>
            text 3
    </tr>
        text 4
</table>
```

Figure 1.1: Faulty HTML Input

it would render the page according to the corrected code shown below:

```
<table border=1 cellpadding=0 cellspacing=0 width=100%>
    <tr valign=middle>
        <td>
            some text
        </td>
    </tr>
    <caption>
        a caption
    </caption>
    <tr valign=middle>
        <td>
            <strong> some more text </strong>
        </td>
        <td>
            text 3
        </td>
    </tr>
    <tr valign=middle>
        <td>
            text 4
        </td>
    </tr>
</table>
```

Figure 1.2: Corrected HTML Input

Notice how missing closures, such as </**strong**>, </**caption**> have been inserted.

Another correction is the addition of missing <**td**> and <**tr**> tags before the string "some text" and closing </**td**> and </**tr**> tags after it. The resulting layout of the page is:

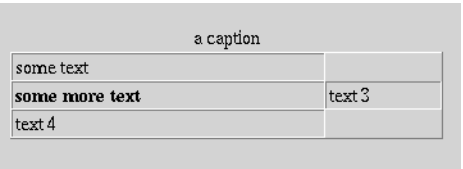


Figure 1.3: VistaPoint Rendering of the Faulty HTML Input

VistaPoint handles Tables and Forms completely.

VistaPoint does progressive image loading. This means that the text of a page can be fully rendered, even though the images are incompletely loaded. The widget can load images on its own, provided they are in local files. For remote images, the application has to fetch the data and feed image data to the widget; it can do so for several images in parallel, i.e., the application can initiate image loading for several images simultaneously, feeding new data to the widget as it becomes available for a given image, then for another image, etc. Image rendering is of very high quality, when the RGBCube and image dithering options are enabled.

VistaPoint accepts the following image formats:

Table 1: Image Formats

Load Image File (remote file)	Load Image Data (local file)
gif	gif
jpeg	jpeg
	xwd
	xpm
	xbm
	bmp

The layout algorithm is such that the widget can display text alongside an image or a table if so directed. Below is an example where the NSL logo is displayed to the left of text that is indented accordingly.

This is the HTML source:

```
<IMG ALIGN="left" SRC="http://www.nsl.fr/logo.gif" width=50
                                                ALT="NSL Logo">
<STRONG>Nova Software Labs</STRONG> provides software
 solutions for the <STRONG>Unix</STRONG> and
 <STRONG>Windows</STRONG> developer. The main products
 include N.S.L.'s own bestselling
<A HREF="/ANG/products.html"> <STRONG>XFaceMaker
</STRONG></A>
line of <STRONG>GUI</STRONG> development tools,
<A HREF="/ANG/products.html"><STRONG>XDrawMaker</STRONG>
</A>, our dedicated editor
```

```
of animated drawings,
```

and this is the resulting display:

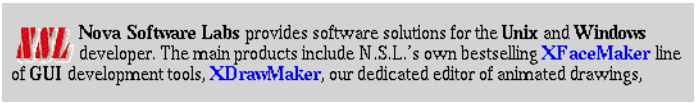


Figure 1.4: Displaying Text Next to an Image

VistaPoint implements some of the new features defined in HTML version 4.0, in particular the handling of Frames. Thus, the widget can read, interpret and display HTML pages that include frames.

Another feature that VistaPoint implements, starting with version 1.2, is WYSIWYG PostScript printing. The widget recognizes a special tag, **<BRPAGE>** that instructs it to start a new page. It has resources to let you specify a page size and page orientation and can render an HTML page with page breaks materialized on the screen. You can control how images that straddle a page break are to be rendered: they can be cut, wherever the page break occurs, or moved to the top of the next page. A PostScript output function lets you specify the page numbers you want to save in the PostScript file.

You will find additional details on the performance and possibilities of VistaPoint in this manual.

For information on HTML, consult the World Wide Web, in particular the W3 pages at **`http://www.w3.org`**

1.1 Document Organization

The installation procedure for the VistaPoint library is described in a separate document.

The VistaPoint reference pages are in **Chapter 2, The XnslHtml Class**. This is where a developer will find information on the widget's resources, its callbacks and callback structures, the constants it defines, etc. This chapter is organized along the same lines as the Motif widget reference pages.

Chapter 3, Creating an Application explains what you should do to create an application that uses VistaPoint, either on its own, or with XFaceMaker as your GUI development tool.

Chapter 4, VistaPoint Program Examples gives tips on how to use the examples provided in the **demos** directory (if the demos have been installed).

The functions that are part of the VistaPoint widget library are described in **Chapter 5, VistaPoint Functions**.

The rest of this chapter describes how the widget library is protected.

This manual assumes that you are familiar with the X Window System and the Motif widget set and that you have access to the Motif and XtIntrinsics reference manuals.

1.2 Library Protection

The VistaPoint widget library distribution includes two copies of the library:

- development library - whenever the widget is instantiated, it requests a license from the NSL network license server. To use this version, you must have a configured license file and the license server daemon, **NSLlicense**, must be running. Please refer to the installation manual for information on configuring and using the NSL license server. Any application using the protected library should free the VistaPoint license before exiting. To do so, call the function ***XnslHtmlComplete***.
- run-time library - this is delivered in encrypted form. To have access to the unprotected library, you must obtain a library decrypting code, once you have purchased the VistaPoint widget. Please refer to the installation manual for information on decrypting the library. With the run-time library, the widget can be used independently of any license file or license server, it no longer requests a license. Applications built with this library can run on any binary compatible host. Applications built with the run-time library can continue to call the ***XnslHtmlComplete*** function, it will do nothing,

1.3 Product Distribution

The product distribution includes several examples of applications built with the widget. We highly recommend that these demos be installed along with the product: the source code of these demos is a good place to look for ideas and examples of resource settings, callbacks, function calls.

CHAPTER 2: The XnslHtml Class

2.1 Synopsis

#include <Xnsl/Html.h>

2.2 Classes

The **XnslHtml** class inherits resources and behavior from the **Core**, **Composite**, **Constraint**, and **XmManager** classes

The class pointer is **xnslHtmlWidgetClass**.

The class name is **XnslHtml**.

2.3 Description

The VistaPoint widget is designed to read, interpret and display HTML[1] version 3.2 text from a buffer provided by the application via the resource **XnslNhtmlBuffer**.

To load a page in the widget, the application should proceed as follows:

1. set the widget's **XnslNhtmlUrl** resource. The URL[2] is the location of the page which is about to be provided to the widget for loading. The widget uses this resource to set its **XnslNhtmlCurrentDirectory** and **XnslNhtmlRootUrl** resources in order to be able to generate well formed URL's when and if it needs to fetch images.
2. fetch the HTML page according to the URL and load it in a buffer.
3. pass the page buffer to the widget by setting the **XnslNhtmlBuffer** resource. If the Html page provided contains no images or applets, the widget parses the page and displays it. If the page contains applets, the widget invokes the **XnslNhtmlScriptAppletCallback** for each applet as it encounters it while parsing. If the page contains images, the widget's behavior depends on the value of the **XnslNhtmlLoadImagePolicy** resource and on the image's URL. The widget can ignore an image, load it on its own, or invoke the **XnslNhtmlLoadImageCallback** to let the application know it needs image data[3]. See section 2.3.1, Loading Images for details
4. When the widget returns control from the XtSetValues call for the **XnslNht-**

1. HTML: Hyper Text Markup Language
2. URL: Uniform Resource Locator

mlBuffer resource, it has taken a copy of the buffer and has obtained enough data to start displaying the page. The application may free its buffer at this time, as the widget has made its own copy of the data. The widget may still need image data, though; if so, it will invoke the **XnslNhtmlLoadImage-Callback** to inform the application it needs image data. Note that the widget will invoke the **XnslNhtmlLoadImageCallback** *before* it returns control from the XtSetValues call, if it needs image data from the application in order to start displaying the page.

VistaPoint has resources to control how pages will be rendered:

- **XnslNhtmlFontSize** specifies the base size of the font used to display the page.
- **XnslNhtmlFontSizeIncrement** specifies the increment in pixels applied to the base font size when the font size is changed by one unit in the HTML text.
- **XnslNhtmlDitherImage** specifies whether images should be displayed with or without dithering.
- **XnslNhtmlRgbCubeDim** specifies if colors should be taken from an RGB cube and, if so, the resource gives the size of the RGB cube.

The best performance in speed and quality for the display of images is obtained with a reasonably sized RGB cube (5 is a good size) and dithering enabled. When the application creates a VistaPoint widget having specified a non-zero value for the **XnslNhtmlRgbCubeDim** resource, the widget attempts to allocate the colors for the RGB cube. The number of colors needed is the dimension cubed (125 colors for a dimension of 5). If the widget can't allocate all the colors, it gives up, and sets the resource to 0, indicating it is working without an RGB cube. To make sure the RGB cube was successfully created, the application should read the value of the resource after setting it. If it is 0, and the application still wants to work with an RGB cube, it should try a smaller size for the RGB cube, until the widget succeeds in creating the cube. Once the RGB cube has been created, the **XnslNhtmlRgbCubeDim** resource can no longer be changed.

VistaPoint has a resources to enable/disable scrolling with mouse button 3:

- **XnslNhtmlDoHandMove** specifies if the user can use mouse button 3 motion to scroll the view window of the page being displayed.

3. Make sure that none of the widget's resources are set during this step, e.g. in the functions called by the XnslNhtmlScriptAppletCallback or the XnslNhtmlLoad-ImageCallback: the widget is doing the XtSetValues for the XnslNhtmlBuffer resource.

2.3.1 Loading Images

The **XnslNhtmlLoadImagePolicy** resource controls the widget's behavior when it encounters an image while parsing the page. Acceptable values for this resource, and resulting behavior are:

- **XnslHTML_DISCARD_IMAGE -** VistaPoint ignores all images. Only the page text will be displayed, images will be skipped.
- **XnslHTML_LOAD_IMAGE_FILE** - VistaPoint loads any local images on its own (URL's of type **`file:/---/<filename>`**). It reads the image data, and renders the image according to the **XnslNhtmlDitherImage** and **XnslNhtmlRgbCubeDim** resources. If it encounters remote images (URL's of type **`ftp:/`** , **`http://`** ,...) it informs the application it needs image data by invoking the **htmlLoadImageCallback** as described below.
- **XnslHTML_CALL_LOAD_IMAGE** - VistaPoint always informs the application that it needs image data, regardless of the image location (provided the image has not been cached - see below). This lets the application control whether a given image should be loaded or skipped. All it needs to do is set the *process_image* field of the callback structure to false.

The image formats that VistaPoint can load on its own are: ***xwd, gif, jpeg, xpm, xbm, bmp***. The image formats that can be loaded through the *XnslDoRemoteImage* function are ***gif*** and ***jpeg***.

2.3.1.1 Communicating with the Application

When VistaPoint is about to load an image and needs data from the application, it invokes the **htmlLoadImageCallback**, passing the image's URL in the callback structure.

The application has the option to skip the image: to do so, it just needs to make sure the *process_image* structure element is set to false (the default).

If the application wants to load the image, it should set the *process_image* structure element to true, fetch the image data from the URL and give the data to the widget by calling the function *XnslDoRemoteImage.* The function returns -1 as long as it has insufficient data to start rendering the image (it does not know the image size). The application should continue to provide data until the function returns 0 or 1. The function returns 1 when the widget has all the data it needs to render the image: the application has finished its job for that image. The function returns 0 when the widget has sufficient data to start rendering the page but will need additional data to fully

render the image. The application will have to call the *XnslDoRemoteImage* function again for that URL, when it has received more data, but can do so outside the **htmlLoadImageCallback.**

2.3.2 Image Cache

Once an image has been loaded, it is cached. An image that is about to be loaded is considered identical to an image in the cache if all the conditions below are true:

- It has the same URL as one of the images in the cache.
- It has the same size as the image in the cache, the size being defined as the optional size specification in the IMG tag, or the absence of such a specification. In other words, the size of an image with a given URL will be considered different every time the width and height specification in the IMG tag differs. For example, given the set of image tags below, the image will be loaded and placed in the cache three times: lines 1, 2, and 4 specify different image sizes. The image for the tag in line 3 will be fetched from the cache, as it is the same as that in line 1.

```
1:   <IMG SRC="file:/usr/local/myimage.gif" WIDTH=10 HEIGHT=20>
2:    <IMG SRC="file:/usr/local/myimage.gif">
3:   <IMG SRC="file:/usr/local/myimage.gif" WIDTH=10 HEIGHT=20>
4:   <IMG SRC="file:/usr/local/myimage.gif" WIDTH=30 HEIGHT=30>
```

- The **XnslNhtmlRgbCubeDim** resource value is the same: all the widgets in an application use the same cache.
- The **XnslNhtmlDitherImage** resource value is the same: all the widgets in an application use the same cache.
- The **visual** is the same: all the widgets in an application use the same cache.

For each image in the cache, a counter keeps track of the number of references to it in the pages currently loaded by the application. An image can be removed from the cache only if it is not in use, i.e. if the reference counter for that image is zero. Two functions let the application remove images from the cache:

- *XnslHtmlRemoveImageFromCache* removes an image from the cache provided its counter is zero (the image is not currently referenced) and does nothing otherwise.
- *XnslHtmlDeleteImageFromCache* removes an image from the cache if its counter is zero (the image is not currently referenced) and otherwise marks the image for removal as soon as its counter reaches zero (the image is no longer referenced, it is no longer loaded by any of the widgets in the application).

2.3.3 Scrolling

The VistaPoint widget handles its own scrolling regardless of whether its parent is an XmScrolledWindow or not; thus, it is advisable not to place the widget in an XmScrolledWindow, if only because a scrolled window parent would add unnecessary overhead. Only the visible portion of the page is loaded in the widget. As the window is scrolled, the corresponding portion of the page is loaded by VistaPoint.

The **XnslNhtmlScrollbarDisplayPolicy** specifies whether to display the scroll bars. When scroll bar display is enabled, the vertical scroll bar is to the right of the widget, and the horizontal scroll bar is at the bottom.

The application can control page scrolling through the *XnslHtmlScrollTo* function.The user can scroll with the scroll bars or using hand move if it is enabled (**XnslNhtmlDoHandMove** resource).

2.3.4 Frames

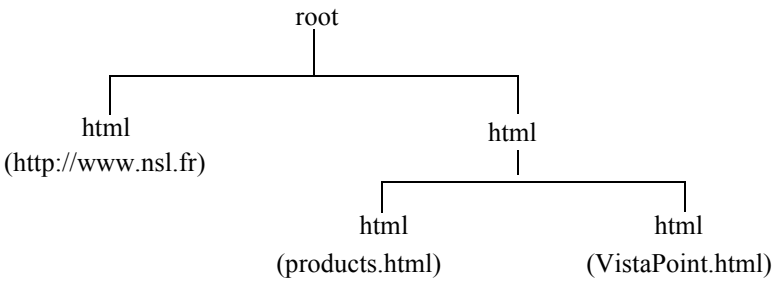
VistaPoint is able to handle frames, as they are defined in HTML version 4.0. Each new frame and each new frame set encountered in a page is rendered with its own instance of the XnslHtml widget. Borders in frames are implemented using XmSeparator widgets. Separators can or cannot be moved, depending on the frame tag arguments.

When loading a page that includes frames, VistaPoint creates additional instances of the XnslHtml widget as needed, in a hierarchy that stems from the main (root) widget.

For example, given the following frame set record:

```
<FRAMESET COLS="30%,*" >
   <FRAME SRC="http://www.nsl.fr" >
   <FRAMESET ROWS="50%,*" >
      <FRAME SRC="http://www.nsl.fr/ANG/products.html" >
      <FRAME SRC="http://www.nsl.fr/ANG/VistaPoint.html" >
   </FRAMESET>
</FRAMESET>
```

the widget would produce the following widget tree (ignoring the separators):



When VistaPoint creates a frame or a frame set widget, it sets the **XnslNhtmlFrameWidget** resource of the instance to True. In the example above, the value of this resource would be *true* for all the VistaPoint widgets in the widget tree except the one labeled *root*.

A frame widget may have a name. This name can subsequently be used as the target element within links: the URL referred to, e.g. in an anchor, will be rendered in the frame whose name has been specified as the target in the anchor.

2.3.5 Callbacks

VistaPoint supports the callbacks listed below. The callback structure returned is the same for all the callbacks. The callback descriptions indicate which of the structure fields are valid for a given callback.

- **XnslNhtmlAbortImageCallback:** triggered by the widget to inform the application that image loading is being aborted due to user interaction and that image data is no longer needed. This happens, for example, when the user clicks on a link to load a new page before all images have been loaded in the current page. *image_data* is passed, to let the application know for which image it should stop providing data.
- **XnslNhtmlActivateCallback**: triggered by releasing mouse button one over a hyperlink. The *href* field and the TITLE field of the hyperlink tag are passed along with the *target* and *node* information.
- **XnslNhtmlFrameCallback**: triggered when the widget creates a new Html widget that will handle the URL associated to a FRAME or FRAMESET tag. The new widget and the URL are passed.
- **XnslNhtmlLoadImageCallback**: triggered by the widget when it needs image data from the application. The image's URL is passed, along with an opaque

structure that the application will use to provide the data it has retrieved. The application should set the *process_image* callback structure element to true if it wants to load the image. The *node* information is passed also if the tag is IMG.

- **XnslNhtmlMouseOverCallback**: triggered by a *MouseMove* action when the mouse pointer is moved over a hyperlink. The *href* field and the TITLE field of the hyperlink tag are passed along with the *node* information.
- **XnslNhtmlPrintFooterCallback**: triggered during postscript output, to let the application specify page footer text. The font used for any footer is Times Roman size 12.
- **XnslNhtmlPrintHeaderCallback**: triggered during postscript output, to let the application specify page header text. The font used for any header is Times Roman size 12.
- **XnslNhtmlScriptAppletCallback**: triggered by the widget when it encounters a SCRIPT or APPLET tag while it is parsing a page. VistaPoint does not know how to interpret scripts or applets on its own. It passes the source code to the application in the callback structure as well as the *node* information.
- **XnslNhtmlSubmitFormCallback**: triggered by a mouse button 1 click on a form's *Submit* button. The form fields and their value are passed.
- **XnslNhtmlSubmitIsindexCallback**: triggered by the user clicking on the button associated to the INDEX tag.
- **XnslNhtmlVisitedLinkCallback**: triggered every time an **<a>** tag is encountered while the page is being parsed. The application should set the *visited* field in the callback structure to 1 if it wants the link to be marked as visited, and to 0 otherwise. A "visited" hyperlink is rendered in the VLINK color, or in the color specified in the **XnslNhtmlDefaultVLinkColor** resource if the link did not specify a VLINK color value.

2.3.6 Controlling Page Size and Printing

VistaPoint has improved layout possibilities that allow for page size specification and page break control. Once a page size is specified (as a standard page name) page beginnings and ends are materialized on the display, along with page numbers. The widget lets you specify whether images that would not fit entirely on a page should be moved to the top of the next page or cut wherever the page break occurs. Finally, VistaPoint recognizes a Break Page tag in the HTML text (BRPAGE), giving the application full control over page breaks. The PostScript output generated by Vi-

staPoint takes all these parameters in account. This means you have WYSIWYG PostScript output (except for page headers and footers; their content is controlled through callbacks, they are not displayed on the screen).

The resources that implement these features are:

- **XnslNhtmlPageSize** specifies a page name used to render the HTML text. Set this resource to do WYSIWYG printing. If a page name is specified (the resource is set to a standard page name, e.g. “a5”, “letter”, “legal”,” a4”, ...), page beginnings and ends are materialized on the screen.
- **XnslNhtmlCutImagePolicy** specifies whether images should be moved to the next page when they straddle a page break.
- **XnslNhtmlPageOrientation** specifies whether page orientation is portrait or landscape.
- **XnslNhtmlPageSizeEndColor** specifies the color used to materialize an end of page on the display.
- **XnslNhtmlPageSizeStartColor** specifies the color used to materialize a beginning of page on the display.

The PostScript output function lets you specify the pages that you want to print with a *from* page and a *to* page argument. Page headers and footers are implemented through callbacks - see below.

2.3.6.1 Controlling Page Breaks Explicitly

VistaPoint recognizes a special tag, **<BRPAGE>** that generates an immediate page break. This tag is recognized in ‘first level’ text; it is ignored if it is placed within a table. With this tag, it is possible to force text and/or images to be rendered at the top of the next page, thus letting you avoid orphan lines in your printing. You need only add a <BRPAGE> tag in the HTML text, where desired. This tag has no effect if no page size is defined for the widget.

2.3.6.2 Specifying Page Headers and Footers

For each page being output to PostScript, the **XnslNhtmlPrintFooterCallback** and the **XnslNhtmlPrintHeaderCallback** are invoked. This lets the application specify the three standard text elements (left, center, and right text) that it wants to output to the page, in the header (**XnslNhtmlPrintHeaderCallback**) and in the footer (**XnslNhtmlPrintFooterCallback**).The application must provide the text in allocated strings in the callback structure. Do not specify static strings, as the widget frees the strings after executing each of the callbacks.

2.3.7 Mouse Interactions

Mouse Button Action	Result
Buttons 1 press on a hyperlink	The *Arm* action is invoked
Button 1 release on a hyperlink	The *Disarm* action is triggered. This invokes the ***activateCallback***
Button 1 motion	The *MouseMove* action is triggered when the pointer is over a hyperlink. This invokes the ***MouseOverCallback***
Button 3 press (with **DoHand-Move** true)	Change cursor to *fleur* cursor
Button3 motion (with **DoHandMove** true)	*HandMove*: scroll page with mouse
Button3 release (with **DoHandMove** true)	Change cursor to normal cursor

2.4 Resources

The **XnslHtml** widget class defines the new set of resources listed in the table below.

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues;

XnslHtml Resource Set

Name Class	Default Type	Access
XnslNhtmlAbortImageCallback XnslCHtmlAbortImageCallback	NULL XtRCallback	C
XnslNhtmlAcceptFrames XnslCHtmlAcceptFrames	TRUE XtRBoolean	CSG
XnslNhtmlActivateCallback XnslCHtmlActivateCallback	NULL XtRCallback	C

XnslHtml Resource Set

Name Class	Default Type	Access
XnslNhtmlAllocColorWithRGB XnslCHtmlAllocColorWithRGB	TRUE XtRBoolean	CSG
XnslNhtmlAnimationSpeedFactor XnslCHtmlAnimationSpeedFactor	2 XtRInt	CSG
XnslNhtmlBlinkRate XnslChtmlBlinkRate	500 XtRInt	CSG
XnslNhtmlBottomMargin XnslCHtmlBottomMargin	10 XtRInt	CSG
XnslNhtmlBuffer XnslCHtmlBuffer	0 XmRString	CS
XnslNhtmlCurrentDirectory XnslCHtmlCurrentDirectory	0 XmRString	G
XnslNhtmlCutImagePolicy XnslCHtmlCutImagePolicy	XnslHTML_ALWAYS_CUT_IMAGE XnslRHtmlCutImagePolicy	CSG
XnslNhtmlDefaultALinkColor XnslCHtmlDefautALinkColor	“blue” XtRPixel	CSG
XnslNhtmlDefaultLinkColor XnslCHtmlDefautLinkColor	“red” XtRPixel	CSG
XnslNhtmlDefaultVLinkColor XnslCHtmlDefautVLinkColor	“green” XtRPixel	CSG
XnslNhtmlDitherImage XnslCHtmlDitherImage	FALSE XtRBoolean	CG
XnslNhtmlDocTitle XnslCHtmlDocTitle	0 XmRString	CSG
XnslNhtmlDoHandMove XnslCHtmlDoHandMove	TRUE XmRBoolean	CSG
XnslNhtmlFixedFontFamily XnslCHtmlFixedFontFamily	“adobe-courier” XmRString	CSG
XnslNhtmlFontSize XnslCHtmlFontSize	14 XtRInt	CSG
XnslNhtmlFontSizeIncrement XnslCHtmlFontSizeIncrement	2 XtRInt	CSG
XnslNhtmlFrameCallback XnslCHtmlFrameCallback	NULL XtRCallback	C

XnslHtml Resource Set

Name Class	Default Type	Access
XnslNhtmlFrameName XnslCHtmlFrameName	NULL XtRString	G
XnslNhtmlFrameRoot XnslCHtmlFrameRoot	NULL XtRWidget	G
XnslNhtmlFrameWidget XnslCHtmlFrameWidget	FALSE XtRBoolean	G
XnslNhtmlHeight XnslCHtmlHeight	1 XnslRHtmlDimension	G
XnslNhtmlHideUnkownTag XnslCHtmlHideUnkownTag	FALSE XmRBoolean	CSG
XnslNhtmlHorizontalScrollBar XnslCHtmlHorizontalScrollBar	NULL XmRWidget	G
XnslNhtmlIndentSpacing XnslCHtmlIndentSpacing	30 XtRInt	CSG
XnslNhtmlLayoutPolicy XnslCHtmlLayoutPolicy	XnslHTML_USE_VISIBLE_WIDTH XnslRHtmlLayoutPolicy	CSG
XnslNhtmlLeftMargin XnslCHtmlLeftMargin	10 XtRInt	CSG
XnslNhtmlLineSpacing XnslCHtmlLineSpacing	0 XtRInt	CSG
XnslNhtmlLoadImageCallback XnslCHtmlLoadImageCallback	NULL XtRCallback	C
XnslNhtmlLoadImagePolicy XnslCHtmlLoadImagePolicy	XnslHTML_LOAD_IMAGE_FILE XnslRHtmlLoadImagePolicy	CSG
XnslNhtmlMinCellPadding XnslCHtmlMinCellPadding	2 XnslRHtmlDimension	CSG
XnslNhtmlMouseOverCallback XnslCHtmlMouseOverCallback	NULL XtRCallback	C
XnslNhtmlNormalCursor XnslCHtmlNormalCursor	0 XtRCursor	CSG
XnslNhtmlNormalFontFamily XnslCHtmlNormalFontFamily	“adobe-times” XmRString	CSG
XnslNhtmlPageOrientation XnslCHtmlPageOrientation	XnslHTML_PORTRAIT XnslRHtmlPageOrientation	CSG

XnslHtml Resource Set

Name Class	Default Type	Access
XnslNhtmlPageSize XnslCHtmlPageSize	0 XmRString	CSG
XnslNhtmlPageSizeEndColor XnslCHtmlPageSizeEndColor	“red” XtRPixel	CSG
XnslNhtmlPageSizeStartColor XnslCHtmlPageSizeStartColor	“green” XtRPixel	CSG
XnslNhtmlPrintFooterCallback XnslCHtmlPrintFooterCallback	NULL XtRCallback	C
XnslNhtmlPrintHeaderCallback XnslCHtmlPrintHeaderCallback	NULL XtRCallback	C
XnslNhtmlRgbCubeDim XnslCHtmlRgbCubeDim	0 XmRInt	CSG[1]
XnslNhtmlRightMargin XnslCHtmlRightMargin	10 XtRInt	CSG
XnslNhtmlRootUrl XnslCHtmlRootUrl	0 XmRString	G
XnslNhtmlScriptAppletCallback XnslCHtmlScriptAppletCallback	NULL XtRCallback	C
XnslNhtmlScrollbarDisplayPolicy XnslCHtmlScrollbarDisplayPolicy	XnslHTML_AS_NEEDED XnslRHtmlScrollbarDisplayPolicy	CSG
XnslNhtmlSource XnslCHtmlSource	0 XmRString	G
XnslNhtmlSubmitFormCallback XnslCHtmlSubmitFormCallback	NULL XtRCallback	C
XnslNhtmlSubmitIsindexCallback XnslCHtmlSubmitIsindexCallback	NULL XtRCallback	C
XnslNhtmlTopMargin XnslCHtmlTopMargin	10 XtRInt	CSG
XnslNhtmlUrl XnslCHtmlUrl	0 XmRString	CSG
XnslNhtmlVerticalScrollBar XnslCHtmlVerticalScrollBar	NULL XmRWidget	G
XnslNhtmlVisitedLinkCallback XnslCHtmlVisitedLinkCallback	NULL XtRCallback	C

XnslHtml Resource Set

Name Class	Default Type	Access
XnslNhtmlWaitCursor XnslCHtmlWaitCursor	"watch" XtRCursor	CSG
XnslNhtmlWidth XnslCHtmlWidth	1 XnslRHtmlDimension	G
XnslNhtmlXOffset XnslCHtmlXOffset	0 XmRInt	G
XnslNhtmlYOffset XnslCHtmlYOffset	0 XmRInt	G

1. Once the XnslNhtmlRgbCubeDim resource has been set to a value that has been accepted (reading it yields a non-zero value) it can no longer be set: the widget's RGBcube has been created.

XnslNhtmlAbortImageCallback

This callback is triggered by the widget to inform the application that image loading is being stopped because the user has initiated loading of a new page, e.g. by clicking on a link. It becomes useless to continue providing image data for the page referenced in the *image_data* element of the structure. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_ABORT_IMAGE
- *event* is set to 0.
- *widget* is the widget that invoked the callback.
- *url* is set to 0.
- *image_data* is the same opaque structure that was passed in the XnslNhtmlLoadImageCallback for that image the first time. The structure is about to be destroyed by the widget, after the callback list has been activated.
- *process_image*: is set to 0.

XnslNhtmlAcceptFrames

If True, specifies that the widget should read, and render the frames it encounters when loading a page. If False, the widget skips any frames it encounters while loading a page.

XnslNhtmlActivateCallback

This callback is triggered when mouse Button 1 is released and the mouse

pointer is over a hyperlink. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_ACTIVATE
- *event* is the event which triggered the callback
- *widget* is the widget that invoked the callback
- *href* is the contents of the *href* field of the hyperlink tag
- *target* specifies the name of the target frame where a document is to be opened. This attribute is defined when frames are being used.
- *title* is the TITLE field of the hyperlink tag
- *detail* can be XnslHTML_HREF, indicating that the tag is an A tag: **<a href=url>** or XnslHTML_IMAGE_MAP, indicating the tag is an image map tag: <**img usemap=mapname>**
- *x* is 0, if detail is XnslHTML_HREF;

  if detail is XnslHTML_IMAGE_MAP it is the x coordinate of the point where the user clicked in the image. The coordinate is given in pixels relative to the top left corner of the image.
- *y* is 0, if detail is XnslHTML_HREF;

  if detail is XnslHTML_IMAGE_MAP it is the y coordinate of the point where the user clicked in the image. The coordinate is given in pixels relative to the top left corner of the image.
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute.

XnslNhtmlAllocColorWithRGB

If True (the default), indicates that colors specified in the page, if any, should be taken from the RGB cube, if it exists (XnslNhtmlRgbCubeDim resource different from 0). If False, or if there is no RGB cube, colors specified in the page are taken from the color map.

XnslNhtmlAnimationSpeedFactor

Specifies the speed dividing factor for animated images.

XnslNhtmlBlinkRate

Specifies the blink rate, in milliseconds, for text tagged with the <blink> tag.

XnslNhtmlBottomMargin

Specifies the margin, in pixels, between the widget's bottom border and the bottom border of the page.

XnslNhtmlBuffer

Specifies the contents of the page that the widget should display. This is straight HTML code; it is provided by the application. This resource can be set, but cannot be read. The widget makes its own copy of the data: the application can free its copy of the buffer once it has set the resource, if it no longer needs it. To read the contents of the widget's current HTML page, the application should *get* the **XnslNhtmlSource** resource. An attempt to *get* the **XnslNhtmlBuffer** resource will yield 0.

XnslNhtmlCurrentDirectory

This is a read only resource. It contains the directory component of the **XnslNhtmlUrl** resource. This resource is used by the widget, along with the **XnslNhtmlRootUrl** resource, to build well formed URL's when it needs to fetch images.

If the value of **XnslNhtmlUrl** is **http://www.nsl.fr/ANG/index.html**, the value of **XnslNhtmlCurrentDirectory** is **http://www.nsl.fr/ANG/**

The widget returns a copy of the string when the application *gets* this resource.

XnslNhtmlCutImagePolicy

Specifies whether images should be moved to the next page when they straddle a page break. Note that, for values other than the default **XnslHTML_ALWAYS_CUT_IMAGE**, images that appear to be next to each other can end up on different pages. This resource is used when generating PostScript and when materializing page breaks on the display. Values can be:

- **XnslHTML_ALWAYS_CUT_IMAGE** - leave all images where they are even if they straddle a page break, rendering the part that fits on the current page and the rest on the next page.
- **XnslHTML_NEVER_CUT_IMAGE** - move any image that straddles a page break to the top of the next page. This applies both to fixed and to floating im-

ages. The only pages that can be cut are those that do not entirely fit within the current page size.

- **XnslHTML_CUT_FLOATING_IMAGE** - move any fixed image that straddles a page break to the top of the next page. Leave floating images where they are, rendering the part that fits on the current page and the rest on the next page.

XnslNhtmlDefaultALinkColor

Specifies the default arm color for a hyperlink text. When the user presses mouse button 1 over a hyperlink text and no **ALINK** color is specified in the page <BODY>, the hyperlink text is drawn with the color specified in this resource.

XnslNhtmlDefaultLinkColor

Specifies the default color for a hyperlink text. If no **LINK** color is specified in the page <BODY>, the hyperlink text is drawn with the color specified in this resource.

XnslNhtmlDefaultVLinkColor

Specifies the default color for a hyperlink text that has been visited. If no **VLINK** color is specified in the page <BODY>, a visited hyperlink text is drawn with the color specified in this resource.

A hyperlink has to be marked as visited by the application which can do this in the **XnslNhtmlVisitedLinkCallback.** The widget has no way of knowing, on its own, that a hyperlink has been visited.

XnslNhtmlDitherImage

If True, the images are dithered according to the Floyd Steinberg algorithm[4]. Image loading is slower with dithering than without, when no RGB cube has been created. Images loaded with dithering have colors that differ less from the original than when dithering is not enabled. Dithering does not slow image loading much when the colors are fetched in the RGB cube.

XnslNhtmlDocTitle

Specifies the title of the current document. The widget returns a copy of the string when the application *gets* this resource.

4. With dithering, the difference between the color effectively assigned and the color requested is propagated to the neighboring pixels: color assignment tries to make up for the differences already incurred.

XnslNhtmlDoHandMove

If True, enable scrolling of the page loaded in the widget using HandMove, i.e. with mouse button 3 drag.

XnslNhtmlFixedFontFamily

Specifies the name of the font family used in the document for fixed fonts. This is the font family used for any text that is marked with tags such as CODE or TT.

XnslNhtmlFontSize

Specifies the base size in pixels of the font used to display the text. This is the font size that is used for text whose FONT SIZE in the HTML text is 3.

XnslNhtmlFontSizeIncrement

Specifies the increment in pixels applied to the base font size when the FONT SIZE is changed by one unit in the HTML text.

XnslNhtmlFrameCallback

Because frames, in an HTML page, specify sub-pages, the widget handles FRAME and FRAMESET tags by creating a new Html widget for each sub-page. Each widget thus created receives a URL. This callback is invoked every time a new widget is created.

The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_FRAME
- *widget* is the root widget[5]
- *href* is the URL that is given to the widget being created
- *frame_widget* is the new widget, i.e. the widget that was just created, the one that will handle the URL in *href*.

XnslNhtmlFrameName

If the widget is a frame widget, the name of the frame if a NAME attribute has been specified for the frame. This is the name by which the *target* field of an anchor tag can reach the frame and change its contents. This is a read only resource.

XnslNhtmlFrameRoot

5. Do not attempt to set any of the root widget's resources in this callback as it is invoked while the widget is in the middle of setting its XnslNhtmlBuffer resource.

If the widget is a frame widget, this resource is the widget ID of the root widget, i.e. the widget that created the frame widget while rendering a page containing frame sets and frames. This is a read only resource.

XnslNhtmlFrameWidget

True if the widget is a frame widget, False otherwise. This is a read only resource.

XnslNhtmlHeight

Height of the page in pixels. This is a read only resource.

XnslNhtmlHideUnkownTag

If False (the default), any tag that is not recognized while the text is parsed is displayed as is. If True, unknown tags are not displayed. For example, in the following HTML code:

```
<pre>
     #include <stdio.h>
</pre>
```

`<stdio.h>` is interpreted as a tag, It will be displayed only if this resource is set to False.

Any text that lies between a start and end unknown tag is displayed along with the tags, if this resource is set to False. For example, given the following HTML code:

```
<funnytag> this is an unknown tag </funnytag>
```

The widget will display:

```
<funnytag> this is an unknown tag </funnytag>
```

XnslNhtmlHorizontalScrollBar

Widget ID of the horizontal scrollbar if any. The scroll bar is created automatically by the widget. This is a read only resource.

XnslNhtmlIndentSpacing

Specifies the number of pixels used to indent entities such as lists, ADDRESS and BLOCKQUOTE tags, i.e. any entity that has to be indented. It is the number of pixels between the left edge of normal text and the left edge of indented text.

XnslNhtmlLayoutPolicy

Specifies the layout policy of text within the widget. Values can be:

- **XnslHTML_USE_WIDGET_WIDTH** where the text is displayed in the full widget width - the user may need to use horizontal scrolling to view hidden

text.

- **XnslHTML_USE_VISIBLE_WIDTH** where the text is displayed so as to fit within the view window. This is the default.

XnslNhtmlLeftMargin

Specifies the margin, in pixels, between the widget's left border and the left border of the page.

XnslNhtmlLineSpacing

Specifies the space, in pixels, between lines.

XnslNhtmlLoadImageCallback

This callback is triggered by the widget when it needs data from the application in order to build and display an image[6]. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_GET_IMAGE
- *widget* is the widget that invoked the callback
- *url* is the url of the image that the widget wants to load. Examples of values for *url* are

```
file:/hot/hot/graphics/myimage.gif
http://www.nsl.fr/ANG/icons/logo.gif
```

- *image_data* is an opaque structure. When the application has retrieved image data and wants to communicate the data to the widget, it passes this structure to the *XnslDoRemoteImage* function which will update the structure according to the data.
- *process_image*: the application should set this field to *True* if it wants the widget to process the image. When the callback is invoked, the field is initialized to *False*, i.e. the widget will not load the image unless the application sets *process_image* to *True*. This is similar to the *doit* field of some of the Motif widget callback structures, e.g. the XmText widget.
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute.

XnslNhtmlLoadImagePolicy

6. Do not attempt to set any of the widget's resources in this callback when it is invoked while the widget is in the middle of setting the XnslNhtmlBuffer resource.

Specifies the policy the widget should apply when it encounters an image in the HTML file being processed. Possible values are:

- **XnslHTML_DISCARD_IMAGE** specifies that the image should be ignored, i.e. the widget should not attempt to load it.
- **XnslHTML_LOAD_IMAGE_FILE** specifies that:

  if the image is in a local file, the widget should load it on its own, without invoking the loadImageCallback (e.g. the URL is **file:/---/image.gif**).

  if the image is in a remote location (e.g. the URL is **ftp:/---/image.gif**, or **http://---/image.gif**, or ...) the loadImageCallback is invoked so that the application can fetch the image data and load the image into the widget via the function *XnslDoRemoteImage*.
- **XnslHTML_CALL_LOAD_IMAGE** specifies that the widget should always invoke the loadImageCallback when it encounters an image. The application will always fetch the image data if it wants to load an image, even if the image is local.

XnslNhtmlMinCellPadding

Specifies the minimum value used for cell padding in tables. This overrides any CELLPADDING specification for tables in the page smaller than the value of this resource.

XnslNhtmlMouseOverCallback

This callback is triggered by a mouse motion over a hyperlink. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_MOUSE_OVER
- *event* is the event which triggered the callback
- *widget* is the widget that invoked the callback
- *href* is the contents of the *href* field of the hyperlink tag
- *title* is the TITLE field of the hyperlink tag
- *target* specifies the name of the target frame where a document is to be opened. This attribute is defined when frames are being used.
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute. This field is valid only if the callback is triggered while processing an IMG tag; it is not valid for the processing of background pixmaps.

XnslNhtmlNormalCursor

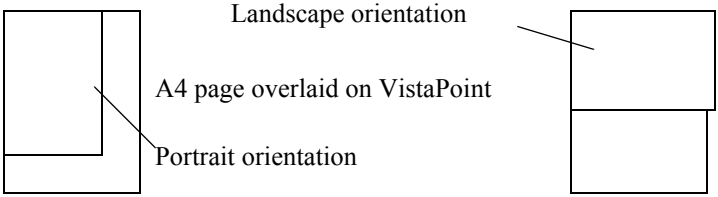
Specifies the name of the cursor used when no interaction is being processed. The default value is 0, i.e. the default cursor.

XnslNhtmlNormalFontFamily

Specifies the name of the font family used in the document for proportional fonts. This is the font family used for any text that does not require a fixed font, i.e. text that is not marked with tags such as CODE or TT.

XnslNhtmlPageOrientation

The value of this resource has an effect only if the **XnslNhtmlPageSize** resource has been set to a value other than zero. When a page size is defined, this resource determines the page orientation, i.e. how the text is rendered in the page, and where automatic page breaks are to be placed.



Acceptable values for this resource are:

- **XnslHTML_PORTRAIT**
- **XnslHTML_LANDSCAPE**

To obtain WYSIWYG PostScript output, specify the same page orientation as in this resource when calling the *XnslHtmlSavePsWidget* function.

XnslNhtmlPageSize

Specifies the page name and, as a consequence, the width/height of the HTML page layout. When this resource is set to **NULL** (the default) the widget's width/height determine the HTML page layout. Valid page names are:

"a3"	"a5"	"b4"	"tabloid"	"legal"
"a4"	"a6"	"b5"	"executive"	"letter"

Table 1: Page Names

If any of these values is specified, VistaPoint renders the HTML text in the page and orientation specified, and materializes page beginnings and ends on the display with lines and page numbers displayed in the colors specified in the **XnslNhtmlPageSizeStartColor** and **XnslNhtmlPageSizeEndColor** resources.

To obtain WYSIWYG PostScript output, specify the same page name as in this resource when calling the *XnslHtmlSavePsWidget* function.

XnslNhtmlPageSizeEndColor

Specify the color used to draw end of page lines and page numbers. Applies only when a page size is defined.

XnslNhtmlPageSizeStartColor

Specify the color used to draw beginning of page lines and page numbers. Applies only when a page size is defined.

XnslNhtmlPrintFooterCallback

This callback is triggered during PostScript output, for each page, to let the application specify the footer strings. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_PRINT_FOOTER
- *widget* is the widget that invoked the callback
- *url* is the URL of the current HTML page
- *page_number* is the number of the page for which a footer is to be specified.
- *left_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the left aligned portion of the footer text for the page. Beware: the string is freed by the widget after callback execution.
- *center_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the centered portion of the footer text for the page. Beware: the string is freed by the widget after callback execution.
- *right_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the right aligned portion of the footer text for the page. Beware: the string is freed by the widget after callback execution.

XnslNhtmlPrintHeaderCallback

This callback is triggered during PostScript output, for each page, to let the application specify the header strings. The following fields of the XnslHtmlCallbackStruct are valid:

- *reason* is XnslHTML_CR_PRINT_HEADER
- *widget* is the widget that invoked the callback
- *url* is the URL of the current HTML page
- *page_number* is the number of the page for which a header is to be specified.
- *left_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the left aligned portion of the header text for the page. Beware: the string is freed by the widget after callback execution.
- *center_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the centered portion of the header text for the page. Beware: the string is freed by the widget after callback execution.
- *right_text* - null when the callback is invoked. This is where the application can provide an allocated string to specify the right aligned portion of the header text for the page. Beware: the string is freed by the widget after callback execution.

XnslNhtmlRgbCubeDim

Specify the size of a side of the RGB cube.[7] For example, if you set this resource to 5, the widget will attempt to allocate 125 colors in the color table: the linear combination of 5 levels of blue, 5 levels of green, 5 levels of red). If it succeeds, the resource is indeed set to 5, colors will be fetched in the RGB cube. If it fails (the required number of colors is not available in the color table) the resource is set to 0, no RGB cube is created. The application should read this resource after setting it, to make sure that the RGB cube was successfully created. If not, try a smaller size.

Once the RGB cube has been created, this resource becomes a read only resource: the size of the RGB cube cannot be changed.

XnslNhtmlRightMargin

Specifies the margin, in pixels, between the widget's right border and the right border of the page.

XnslNhtmlRootUrl

This is a read only resource. It contains the name component of the **XnslNhtmlUrl** resource. This resource is used by the widget, along with the **XnslNhtmlCurrentDirectory** resource, to build well formed URL's when it needs to fetch images.

If the value of **XnslNhtmlUrl** is **http://www.nsl.fr/ANG/index.html**, the value of **XnslNhtmlRootUrl** is **http://www.nsl.fr**/

The widget returns a copy of the string when the application *gets* this resource.

XnslNhtmlScriptAppletCallback

This callback is invoked when the parser encounters a SCRIPT or APPLET tag. The widget does not know how to handle such tags, so it passes the script or the applet *source* to the application [8].

The following fields of the XnslHtmlCallbackStruct are valid:

m *reason* is XnslHTML_CR_SCRIPT if the callback is invoked because of a SCRIPT tag and XnslHTML_CR_APPLET if the callback is invoked because

7. When an image is loaded and an RGB cube has been successfully created, colors are assigned according to the colors available in the RGB cube: pixels whose color is not in the cube, get the color that is closest to the one requested.
8. Do not attempt to set any of the widget's resources in this callback as it is invoked while the widget is in the middle of setting its XnslNhtmlBuffer resource.

of an APPLET tag.

- *widget* is the widget that invoked the callback
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute.
- *source* is the source code of the script or applet.
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute.

XnslNhtmlScrollbarDisplayPolicy

Specifies how to display the scrollbars of the XnslHtml widget; accepted values are:

- **XnslHTML_STATIC** the scrollbars are always displayed.
- **XnslHTML_AS_NEEDED** the scrollbars are displayed only when needed.
- **XnslHTML_NONE** the scrollbars are never displayed. The user can still do *handmove* scrolling if the **XnslNhtmlDoHandMove** resource is set to True.

The application can scroll the page in the widget by calling the *XnslHtmlScrollTo* function, regardless of the setting of **XnslNhtmlScrollbarDisplayPolicy**.

XnslNhtmlSource

This is a read only resource. It is the contents of the page that is currently loaded in the widget. To load a page in the widget, *set* the **XnslNhtmlBuffer** resource. To read the page loaded in the widget, *get* the **XnslNhtmlSource** resource.

The widget returns a copy of the string when the application *gets* this resource.

XnslNhtmlSubmitFormCallback

This callback is triggered when the user clicks on a form's *Submit* button or on a graphical submit button (an input whose type is image: **`<input type=image>`**).

The following fields of the XnslHtmlCallbackStruct are valid:

m *reason* is XnslHTML_CR_SUBMIT_FORM

m *widget* is the widget that invoked the callback

m *action* is the value of the action field of the HTML tag

m *enctype* is the value of the enctype field of the HTML tag

m *method* can be XnslHTML_GET or XnslHTML_POST. The value is read by the parser in the form's definition and placed in the structure field for the application. It specifies how the application should pass the information contained in the form to the server. If *method* is XnslHTML_GET, the application should open a connection to the server and request a URL which may include parameters built from the <name> <value> pairs. If *method* is XnslHTML_POST, the application should open a connection to the server and send the form data directly.

m *name_values* is a linked list of <name> <value> pairs representing the contents of the form. Each field defined in the form is represented by such a <name> <value> pair where *value* depends on the type of field. For texts, *value* is the contents of the text. For other fields, *value* is given in the VALUE field of the INPUT tag.

m *detail* is XnslHTML_SUBMIT_BUTTON if the callback was invoked by clicking on the form's *Submit* button, and XnslHTML_SUBMIT_IMAGE if the callback was invoked by clicking on a graphical submit button.

m *x* is 0 if detail is XnslHTML_SUBMIT_BUTTON

if detail is XnslHTML_SUBMIT_IMAGE, it is the x coordinate of the point where the user clicked in the graphical submit button. The coordinate is given in pixels relative to the top left corner of the button.

m *y* is 0, if detail is XnslHTML_SUBMIT_BUTTON

if detail is XnslHTML_SUBMIT_IMAGE, it is the y coordinate of the point where the user clicked in the graphical submit button. The coordinate is given in pixels relative to the top left corner of the button.

XnslNhtmlSubmitIsindexCallback

An <ISINDEX> tag comprises a text field and a button. The callback is triggered when the user clicks on the button.

The following fields of the XnslHtmlCallbackStruct are valid:

m *reason* is XnslHTML_CR_SUBMIT_ISINDEX

m *widget* is the widget that invoked the callback

m *text* is the contents of the text field associated to the ISINDEX tag.

XnslNhtmlTopMargin

Specifies the margin, in pixels, between the widget's top border and the top border of the page.

XnslNhtmlUrl

Specifies the current page's URL. This resource is used by the widget to set the **XnslNhtmlCurrentDirectory** and **XnslNhtmlRootUrl** resources. These, in turn, are used to build well formed URL's when the widget needs to load images. The application should set this resource *before* setting the **XnslNhtmlBuffer** resource.

The widget returns a copy of the string when the application *gets* this resource.

XnslNhtmlVerticalScrollBar

Widget ID of the vertical scrollbar if any. The scroll bar is created automatically by the widget. This is a read only resource.

XnslNhtmlVisitedLinkCallback

This callback is triggered every time an **<a>** tag is encountered while the page is being parsed. The application should set the *visited* field in the callback structure to 1 if it wants the link to be marked as visited, and to 0 otherwise.

The following fields of the XnslHtmlCallbackStruct are valid:

m *reason* is XnslHTML_CR_LINK_VISITED

m *widget* is the widget that invoked the callback

m *href* is the contents of the *href* field of the hyperlink tag

m *title* is the TITLE field of the hyperlink tag

- *root* is the value of the **XnslNhtmlRootUrl** resource of the widget.
- *url* is the value of the **XnslNhtmlUrl** resource of the widget.
- *visited* is the field that the application can set to 1 to mark the link as having been visited. A link that is marked visited is displayed with the VLINK color if this is specified in the tag, or with the color specified in the **XnslNhtmlDefaultVLinkColor** resource.
- *node* is an opaque structure that the application can pass to the *XnslHtmlGetTagInfo* function to retrieve the *name value* pairs of the tag attribute.

XnslNhtmlWaitCursor

Specifies the name of the cursor used when the widget is busy. Default value is "watch".

XnslNhtmlWidth

Width of the page in pixels. This is a read only resource.

XnslNhtmlXOffset

Specifies the value of the horizontal scrollBar.

XnslNhtmlYOffset

Specifies the value of the vertical scrollBar.

2.4.1 Inherited Resources

XnslHtml inherits the following resources from the named superclass. For a description of each resource, refer to the man page of the superclass.

XmManager Resource Set

Name Class	Default Type	Access
XmNbottomShadowColor XmCBottomShadowColor	dynamic Pixel	CSG
XmNbottomShadowPixmap XmCBottomShadowPixmap	XmUNSPECIFIED_PIXMAP Pixmap	CSG
XmNforeground XmCForeground	dynamic Pixel	CSG
XmNhelpCallback XmCCallback	NULL XtCallbackList	C

XmManager Resource Set

Name Class	Default Type	Access
XmNhighlightColor XmCHighlightColor	dynamic Pixel	CSG
XmNhighlightPixmap XmCHighlightPixmap	dynamic Pixmap	CSG
XmNinitialFocus XmCInitialFocus	dynamic Widget	CSG
XmNnavigationType XmCNavigationType	XmTAB_GROUP XmNavigationType	CSG
XmNshadowThickness XmCShadowThickness	dynamic Dimension	CSG
XmNstringDirection XmCStringDirection	dynamic XmStringDirection	CG
XmNtopShadowColor XmCTopShadowColor	dynamic Pixel	CSG
XmNtopShadowPixmap XmCTopShadowPixmap	dynamic Pixmap	CSG
XmNtraversalOn XmCTraversalOn	True Boolean	CSG
XmNunitType XmCUnitType	dynamic unsigned char	CSG
XmNuserData XmCUserData	NULL XtPointer	CSG

Composite Resource Set

Name Class	Default Type	Access
XmNchildren XmCReadOnly	NULL WidgetList	G
XmNinsertPosition XmCInsertPosition	NULL XtOrderProc	CSG
XmNnumChildren XmCReadOnly	0 Cardinal	G

Core Resource Set

Name Class	Default Type	Access
XmNaccelerators XmCAccelerators	dynamic XtAccelerators	N/A
XmNancestorSensitive XmCSensitive	dynamic Boolean	G
XmNbackground XmCBackground	dynamic Pixel	CSG
XmNbackgroundPixmap XmCPixmap	XmUNSPECIFIED_PIXMAP Pixmap	CSG
XmNborderColor XmCBorderColor	XtDefaultForeground Pixel	CSG
XmNborderPixmap XmCPixmap	XmUNSPECIFIED_PIXMAP Pixmap	CSG
XmNborderWidth XmCBorderWidth	0 Dimension	CSG
XmNcolormap XmCColormap	dynamic Colormap	CG
XmNdepth XmCDepth	dynamic int	CG
XmNdestroyCallback XmCCallback	NULL XtCallbackList	C
XmNheight XmCHeight	dynamic Dimension	CSG
XmNinitialResourcesPersistent XmCInitialResourcesPersistent	True Boolean	C
XmNmappedWhenManaged XmCMappedWhenManaged	True Boolean	CSG
XmNscreen XmCScreen	dynamic Screen	CG
XmNsensitive XmCSensitive	True Boolean	CSG
XmNtranslations XmCTranslations	dynamic XtTranslations	CSG
XmNwidth XmCWidth	dynamic Dimension	CSG

Core Resource Set

Name Class	Default Type	Access
XmNx XmCPosition	0 Position	CSG
XmNy XmCPosition	0 Position	CSG

2.5 Callback Information

The callback structure returned is the same for all callbacks of the VistaPoint widget. The application should first look at the reason field to determine which fields of the structure are valid.

```
typedef struct {
    int                     reason;
    XEvent                  *event;
    Widget                  widget;
    char                    *href;
    char                    *target;
    char                    *title;
    char                    *root;
    char                    *url;
    int                     visited;
    XtPointer               image_data;
    Boolean                 process_image;
    char                    *action; /* Form stuff */
    char                    *enctype;
    XnslHtmlFormMethod      method;
    XnslHtmlNameValue       *name_values;
    char                    *text; /* isindex */
    XnslHtmlActivateDetail  detail; /*map or submit image*/
    int                     x, y;
    XnslHtmlNode            node;
    char                    *source; /* script and applet */
    Widget                  frame_widget; /* frame */
    char                    *frame_name;
    int                     page_number; /*print header & footer*/
    char                    *left_text; /*freed after callback*/
    char                    *center_text;/*freed after callback*/
```

```
    char                    *right_text; /*freed after callback*/
} XnslHtmlCallbackStruct;
```

2.6 Translations

The following translations are defined for the VistaPoint widget.

```
<Btn1Down>:          Arm()
<Btn1Motion>:        Motion()
<Btn1Up>:            Disarm()
<Btn3Down>:          HandArm()
<Btn3Motion>:        HandMove()
<Btn3Up>:            HandDisarm()
<MouseMoved>:        MouseMove()
~Shift ~Meta ~Alt <Key>Tab:  TraverseNext()
 Shift ~Meta ~Alt <Key>Tab:  TraversePrev()
~Shift ~Meta ~Alt <Key>osfUp:    ProcessKey()
~Shift ~Meta ~Alt <Key>osfDown:  ProcessKey()
~Shift ~Meta ~Alt <Key>osfLeft:  ProcessKey()
~Shift ~Meta ~Alt <Key>osfRight: ProcessKey()
```

2.7 Actions

The following actions are defined for the VistaPoint widget:

m *Arm()*

- If the event is over a hyperlink text, repaint the text with ALINK color.

m *Disarm()*

- Trigger the activateCallback if the event occurred over a hyperlink. Change cursor to normal cursor.

m *HandArm()*

- Change cursor to *fleur* cursor.

m *HandDisarm()*

- Change cursor to normal cursor.

m *HandMove()*

- Scroll widget in viewing window.

m *Motion()*

- Repaint hyperlink text with ALINK color or default color, depending on whether motion is towards or away from a hyperlink text.

m *MouseMove()*

- Trigger the mouseOverCallback when the mouse pointer is moved over a hyperlink. Change cursor to *hand* cursor.

m *TraverseNext()*

- Traverse to next tab group.

m *TraversePrev()*

- Traverse to previous tab group.

2.8 Constants

The values for the constants defined for the VistaPoint widget are listed below:

2.8.1 Constants for resources

- XnslNhtmlCutImagePolicy
  - XnslHTML_ALWAYS_CUT_IMAGE
  - XnslHTML_NEVER_CUT_IMAGE
  - XnslHTML_CUT_FLOATING_IMAGE
- XnslNhtmlLayoutPolicy
  - XnslHTML_USE_VISIBLE_WIDTH
  - XnslHTML_USE_WIDGET_WIDTH
- XnslNhtmlLoadImagePolicy
  - XnslHTML_DISCARD_IMAGE
  - XnslHTML_LOAD_IMAGE_FILE
  - XnslHTML_CALL_LOAD_IMAGE
- XnslNhtmlPageOrientation
  - XnslHTML_PORTRAIT
  - XnslHTML_LANDSCAPE
- XnslNhtmlScrollbarDisplayPolicy
  - XnslHTML_STATIC
  - XnslHTML_AS_NEEDED
  - XnslHTML_NONE

2.8.2 Constants for the callback structure

- Callback reason:
    - XnslHTML_CR_ABORT_IMAGE
    - XnslHTML_CR_ACTIVATE
    - XnslHTML_CR_APPLET
    - XnslHTML_CR_FRAME
    - XnslHTML_CR_GET_IMAGE
    - XnslHTML_CR_LINK_VISITED
    - XnslHTML_CR_MOUSE_OVER
    - XnslHTML_CR_PRINT_FOOTER
    - XnslHTML_CR_PRINT_HEADER
    - XnslHTML_CR_SCRIPT
    - XnslHTML_CR_SUBMIT_FORM
    - XnslHTML_CR_SUBMIT_ISINDEX
- Values of XnslHtmlActivateDetail in the callback structure:
    - XnslHTML_HREF
    - XnslHTML_IMAGE_MAP
    - XnslHTML_SUBMIT_BUTTON
    - XnslHTML_SUBMIT_IMAGE
- Values for XnslHtmlFormMethod the field in the callback structure:
    - XnslHTML_GET
    - XnslHTML_POST

2.9 Structures

The structures that are defined for the VistaPoint widget are:

- **XnslHtmlArg** structure - a member of the XnslHtmlTagInfo structure:

```
typedef struct _XnslHtmlArg {
    char                *name;
    char                *value;
    unsigned long       offset;
    unsigned long       len;
    struct _XnslHtmlArg *next;
} XnslHtmlArg;
```

- **XnslHtmlFrameInfo** structure - returned by the XnslHtmlGetFrameInfo function:

```
typedef struct _XnslHtmlFrameInfo {
    char                        *name;
    char                        *url;
    Widget                      w;
    struct _XnslHtmlFrameInfo   *children;
    int                         num_children;
} XnslHtmlFrameInfo;
```

- **XnslHtmlNameValue** structure - a member of the callback structure:

```
typedef struct  _XnslHtmlNameValue {
    char                       *name;
    char                       *value;
    char                       **values;
    struct _XnslHtmlNameValue  *next;
} XnslHtmlNameValue;
```

Note on *value* and *values*: If the callback needs to return a single value, it does so in the *value* field. If it needs to return several value strings, it returns a zero terminated String table in the *values* field. Thus, applications should first test if the *value* field

has data and, if the field is NULL, it should fetch the values in the *values* String table, and test for a NULL string that signals the end of the table has been reached. The callback will never return data in both the *value* and the *values* fields.

- **XnslHtmlTagInfo** structure - returned by the XnslHtmlGetTagInfo function:

```
typedef struct _XnslHtmlTagInfo {
    char                        *name;
    unsigned long               offset;
    unsigned long               len;
    char                        *text;
    XnslHtmlArg                 *args;
    struct _XnslHtmlTagInfo     *next;
} XnslHtmlTagInfo;
```

CHAPTER 3: Creating an Application

This chapter describes what you need to do to build applications that use the VistaPoint library.

You can build an application without using any development tool, just making calls to the XnslHtml Library functions, the Xt functions and the Xm functions. Or, if you have XFaceMaker[1], you can build your interface file with the tool and set the VistaPoint widget resources with the resource editor, then build a compiled or interpreted application.

3.1 VistaPoint Library Alone

To build an application directly with the VistaPoint Library, you have to

1. Initialize the toolkit: **`toplevel = XtAppInitialize( ... )`**
2. Write the application and interface code. The interface code will have to instantiate the widget, set resources, include calls to the XnslHtml Library functions.
3. If you are using the development library, write an exit function and a Close Handler in which you free the VistaPoint widget license: **`XnslHtmlComplete()`** This function does nothing when a non-protected run-time library is used so that it is good practice to always call XnslHtmlComplete when exiting an application linked with the VistaPoint library.
4. Compile the code and link the object code with any application connected libraries, and the following libraries:

   XnslHtml, Xm, Xt, Xext, X11

If you are using X11R6, you will have to link with additional libraries such as ICE and SM.

You may need to add some system libraries such as PW, gen, malloc, and so on depending on your environment. You may also have to specify compilation and link flags depending on your platform, your compiler, your linker.

1. Or, if you have a different GUI Builder that supports third party widget extensions, you can include the XnslHtml in your tool and work from there.

Your command line will look something like this:

```
cc -o myprog myprog.c -lXnslHtml -lXm -lXt -lXext -lX11
```

You must place the XnslHtml library before any libraries other than the application specific libraries.

3.2 VistaPoint Library and XFaceMaker

To build applications with XFaceMaker using the VistaPoint widget, you have to use an extended XFaceMaker executable in which you have loaded the XnslHtml extension.

To do so, launch **xfm** (or any XFaceMaker executable); in the ***File*** menu, select ***Load Extensions...*** and select the XnslHtml extension and any other extensions you want to include in your XFaceMaker executable, e.g. the XnslTree widget. Please refer to the XFaceMaker manuals for further detail.

When you use XFaceMaker to build your application, you can choose two execution modes for the interface of your application.

1. Execute the interface in interpreted mode.
2. Execute the interface in compiled mode.

To build such applications the Makefile generated by a given XFaceMaker executable refers to the include files and the library files of all entities included in the extensions *regardless of whether a particular interface makes any reference to the corresponding widgets or functions*.

Applications built with an extended XFaceMaker are linked with additional libraries:

Table 3.1: Basic Extended XFaceMaker Libraries

Library	Link flag	Description
XnslStand	-lXnslStand	Common widget support code for XFaceMaker
XnslCtrl	-lXnslCtrl	Control widgets: XnslBarGraph, XnslIndicator, XnslJoystick, XnslMeter, XnslSlider, XnslStepper
XnslFm	-lXnslFm	XFaceMaker support

- **Interpreted Interface**

Generally, you will use interpreted mode for the interface while you are developing your application: in interpreted mode, changes made in the interface file (.fm file) are taken into account without your having to recompile the interface, or relink the application. This is because, in interpreted mode, the application just loads the .fm file.

To build an application that runs in interpreted mode proceed as follows:

1. With XFaceMaker, build your interface. Set the VistaPoint widget resources as desired.
2. Make sure your interface includes a Quit or Exit button. In the button's activateCallback, call **`XnslHtmlComplete()`** to free the VistaPoint widget license. You should also add a Close Handler which calls the XnslHtmlComplete function.
3. Save your interface as a .fm file, generate the application files using XFaceMaker (Save Appli Files)
4. If your interface calls application functions, add the corresponding code in the *xyz*App.c file (where *xyz* stands for the name of your interface file), or modify the Makefile generated (or the MakeFile pattern) as needed in order to link with the appropriate application code.
5. Compile with the command

```
make -f xyzMake interpreted
```

where xyz stands for the name of your interface file.

In interpreted mode, the application is linked with the Fm library of XFaceMaker and any libraries required by the extensions with which the extended XFaceMaker you are using was built. Among these, you will have XnslFm, XnslCtrl, XnslStand and, in our case, XnslHtml.

- **Compiled Interface**

If you generate the C code for your interface any changes you make in your interface will require that you rebuild your application. You will generally build applications in compiled mode when your development is well under way or finished, and when you want to deploy your application on a platform where XFaceMaker is not accessible.

To build an application that runs the interface in compiled mode proceed as follows:

1. With XFaceMaker, build your interface. Set the VistaPoint widget resources as desired.
2. Make sure your interface includes a Quit or Exit button. In the button's activateCallback, call **`XnslHtmlComplete()`** to free the VistaPoint widget license. You should also add a Close Handler which calls the XnslHtmlComplete function.
3. If needed, set XFaceMaker's C code generation options.
4. Save your interface as C code, generate the application files using XFaceMaker (Save Appli Files)
5. If your interface calls application functions, add the corresponding code in the *xyz*App.c file (where *xyz* stands for the name of your interface file), or modify the Makefile generated (or the MakeFile pattern) as needed in order to link with the appropriate application code.
6. Compile with the command

```
make -f xyzMake compiled
```

   where xyz stands for the name of your interface file.

In compiled mode, the application is linked with the Fm_c library of XFaceMaker and any libraries required by the extensions with which the extended XFaceMaker you are using was built. Among these, you will have XnslStand, XnslCtrl and, in our case, XnslHtml.

CHAPTER 4: VistaPoint Program Examples

Several applications built with the VistaPoint widget are included in the product distribution. Do not hesitate to examine the source code that comes with these applications. It should help you understand how to set the widget resources and program your callback functions. It will also give you examples of calls to the various functions documented in this manual.

You are free to use any ideas you find in these sources while developing your applications, provided you acknowledge their origin.

4.1 Cexample -Simple Contextual Help

This application illustrates how VistaPoint can be used to implement contextual help.

To launch the application, first make sure that either the NSLHOME or the CEXAMPLEHOME environment variable is set properly:

NSLHOME, if defined, must be set to the NSL product installation directory.

If NSLHOME is not defined, CEXAMPLEHOME must be set to the directory where the cexample demo was installed.

Change to the directory:

```
$NSLHOME/demos/vistapoint/cexample
```

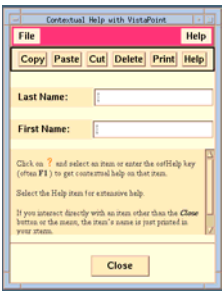
or to the directory:

```
$CEXAMPLEHOME
```

and launch the executable

```
./cexample
```

Your screen should look like this:



This application is built directly with Xt and Motif calls, without using XFaceMaker or any other interface builder. It illustrates the basic functions of the VistaPoint widget. This is the simplest of the VistaPoint demos.

The interface is built around a VistaPoint widget and a few command widgets whose sole purpose is to provide elements on which to implement contextual help. The contents of the VistaPoint widget is initialized through the **XnslNhtmlBuffer** resource. The **XnslNhtmlActivateCallback** is invoked when the user clicks on the "***?***" link. This changes the cursor to the question mark cursor and sets a global HelpOn variable. As the user moves the cursor to one of the command widgets, and clicks, another VistaPoint widget (initialized with the `./help.html` file) is popped and the help text corresponding to that element is displayed.

Pressing the ***osfHelp*** key (often ***F1***) provides the same help text, on the command that has the keyboard focus, i.e. that received the keyPress event.

The ***Help*** menu has an ***About this demo ...*** item that pops a window with a VistaPoint widget that loads the `./about.html` file.

To exit this application, click the ***Close*** button or the ***Quit*** item in the ***File*** menu.

4.2 Forge - Interface Control

This application illustrates how VistaPoint can be used to control an application through an HTML form, and to implement contextual help.

To launch the application, first make sure that either the NSLHOME or the FORGEHOME environment variable is set properly:

NSLHOME, if defined, must be set to the NSL product installation directory.

If NSLHOME is not defined, FORGEHOME must be set to the directory where the forge demo was installed.

Change to the directory:

```
$NSLHOME/demos/vistapoint/forge
```

or to the directory:

```
$FORGEHOME
```

and launch the executable

```
./htmlforge
```

You can specify one of the following command options:

- **-owncmap** - Make **forge** use its own colormap.

- **-urc** - Use RGBCube: colors are allocated in an rgb cube, allowing faster color allocation
- **-help** - List command line options available and exit.

Your screen should look like this:

This application is built with XFaceMaker. The interface includes:

- a MenuBar.
- an XmLabel to display what the program generates.
- a VistaPoint widget.
- a scrolled text to display status information during image calculation.

The three windows accessible through the ***Help*** item in the MenuBar are also built around a VistaPoint widget.

4.2.1 The Main Window

The starting point of this application is the **ppmforge** program developed by John Walker of Autodesk SA. The original program generates fractal images of planets, starry skies, or clouds according to the parameters with which it is launched.

The modified version we present here is interactive. Its graphic interface makes extensive use of HTML and of the VistaPoint callback mechanism.

Forge uses frames in VistaPoint. The HTML page that is loaded in the VistaPoint widget includes two frames in a frameset (file `main.html`). The upper frame contains an HTML form. This is where the user can specify the image generation parameters. The lower frame will display the image generated when the user *submits* the form by clicking on ***On Screen!***

User Interaction in the Main Window

- ***Specifying Image Parameters*** - To modify the image parameters, enter new values in the HTML form. Once you are happy with your settings, click on ***On Screen!*** to *submit* the form. Click on Reset to return all parameters to their default value. Using a form to provide data to the application illustrates how, thanks to VistaPoint's callbacks, it is possible to use standard HTML mechanisms to drive an application that requires data input from the user.

  The **XnslNHtmlSubmitFormCallback** is invoked by VistaPoint when the form is submitted. The form fields and their values are passed to the application in the callback structure. The application function has all the information needed to generate a new image and display it in the lower frame.

- ***Mouse Motion*** - As you move the mouse pointer over sensitive text fragments (the underlined words), a short explanation of the meaning of the parameter associated to the word is displayed in a small popup box.

  This is implemented with the **XnslNHtmlMouseOverCallback**: when the mouse is over a hyperlink, here an **A** tag, the XnslNHtmlMouseOverCallback is invoked by the widget. The callback structure, with the *href* and *title* fields, is passed to the application function attached to the callback. The function displays the contents of the *title* field in the small box. It does not use the other fields. The HTML text used for this is in the `forge.html` file.

- ***Button 1 Click*** - When you click mouse button 1 over a sensitive text fragment (underlined word) an extensive explanation of the meaning of the parameter associated to the word is displayed in a popup box that remains on the screen until you click again, anywhere on the screen.

  This is implemented with the **XnslNHtmlActivateCallback**: on a mouse button 1 click over a hyperlink, here an **A** tag, the XnslNHtmlActivateCallback is invoked by the widget. The callback structure, with the *href*, *title*, and *node* fields is passed to the application function attached to the callback. The function displays the contents of the *href* field in the popup box.

4.2.2 The Help Windows

The Help menu lets you open three windows.

- ***The About Window*** loads an HTML page into a VistaPoint widget. This is a simple illustration of how the application can control scrolling in the VistaPoint widget, through its convenience functions. To suspend scrolling, press the ***Hold*** button. Scrolling resumes when release the button. To close the window, click on ***OK***.
- ***On Ppmforge*** loads the man page for ppmforge into a VistaPoint widget. The text is the original man page for the program, converted to HTML format. The sole purpose of this window is to provide information on the ppmforge program, it does not illustrate any specific features of the widget other than standard HTML text handling as it is done by VistaPoint.
- ***On HtmlForge*** loads, in a VistaPoint widget, an HTML page in which a client side image map has been defined. The image map defines three horizontal bands overlaid on the representation of the main window. Click on one of the bands to scroll the page to the (succinct) explanation of the area's functionality. The purpose, here, is not to provide detailed help on our application, but to illustrate one of the ways HTML can be used to implement efficient help.

4.3 HTEarth - Interface Control

This application illustrates how VistaPoint can be used to control an application from an HTML page, and to implement contextual help.

The widget invokes callbacks in connection with hyperlinks, and passes the associated tag information in its callback structure. It is possible to add information in the tags of an HTML page that is ignored by an HTML parser. This information can be retrieved in the functions you attach to the callbacks, and specify behavior in the application. Thus, just by adding information in your hyperlink tags, you can use the VistaPoint widget to drive an application.

To launch the application, first make sure that either the NSLHOME or the HTEARTHHOME environment variable is set properly:

NSLHOME, if defined, must be set to the NSL product installation directory.

If NSLHOME is not defined, HTEARTHHOME must be set to the directory where the htearth demo was installed.

Change to the directory:

```
$NSLHOME/demos/vistapoint/htearth
```

or to the directory:

```
$HTEARTHHOME
```

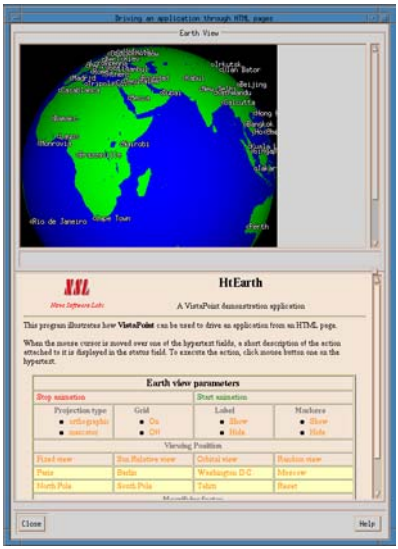
and launch the executable

```
./htearth
```

You can specify one of the following command options:

- **-owncmap** - Make **htearth** use its own colormap.
- **-urc** - Use RGBCube: colors are allocated in an rgb cube, allowing faster color allocation
- **-verbose -** Output connection and status messages in the xterm window during execution. Since the connection is made through the **wget** program, most of the output is from **wget**.
- **-help** - List command line options available and exit.

Your screen should look like this:



This application is built with XFaceMaker. The interface includes:

- an XmPanedWindow.
- in the upper pane, an XmDrawingArea displayed in an XmScrolledWindow with, below it, an XmTextField in which status information is output.
- In the lower pane, a **VistaPoint** widget with HTML text with tag attributes that will trigger actions as explained below.

- at the bottom of the lower pane, a ***Close*** button lets you exit the application; a ***Help*** button opens a Help window.

The Help window includes:

- an XmPanedWindow.
- in the upper pane, a **VistaPoint** widget with HTML text that defines sensitive areas in view of displaying MS Windows type tips.
- in the lower pane, a VistaPoint widget with standard HTML text describing the application.
- at the bottom of the lower pane, a button to close the Help window, and a button to open a Copyright window.

The Copyright window includes:

- a **VistaPoint** widget which contains HTML text. The ***Close*** field is not a separate button, here; it is implemented as a hyperlink within the HTML text.

4.3.1 The Main Window

The starting point of this application is the **xearth** program developed by *Kirk Lauritz Johnson, Jim Frost and Jamie Zawinski*; the source code is available at:

```
http://cag-www.lcs.mit.edu/~tuna/xearth/index.html
```

The original program calculates a view of the earth from a point of view that you provide as a parameter. The program was designed to generate nice screensavers or root pixmaps and not for interactive use.

In **htearth**, the earth view is displayed in the XmDrawingArea in the upper pane of the paned window, according to the parameters specified in the lower pane.

User Interaction in the Main Window

- ***Mouse Motion*** - As you move the mouse pointer over sensitive text fragments, the underlying action (that you would trigger if you clicked on the text) is displayed in the status field below the drawing area.

  This is implemented with the **XnslNHtmlMouseOverCallback**: when the mouse is over a hyperlink, here an **A** tag, the XnslNHtmlMouseOverCallback is invoked by the widget. The callback structure, with the *href* and *title* fields, is passed to the application function attached to the callback. The function displays the contents of the *title* field in the status text field. It does not use the other fields. For example, the **A** tag for *mercator* is:

  ```
  <a href="Projection type set to mercator"
  title="Set the projection type to mercator"
  projection=mercator>
  mercator
  </a>
  ```

  You can verify that, when the mouse pointer is moved over *mercator*, the text associated to *title* is displayed.

- ***Button 1 Click*** - When you click mouse button 1 on sensitive text, you set the earth view parameter to the value in the text. Note that value settings are applied only when the animation is ON, i.e. when new frames are being calculated.

  This is implemented with the **XnslNHtmlActivateCallback**: on a mouse button 1 click over a hyperlink, here an **A** tag, the XnslNHtmlActivateCallback is invoked by the widget. The callback structure, with the *href*, *title*, and *node* fields is passed to the application function attached to the callback. The function displays the contents of the *href* field in the status field below the drawing area. It uses the *node* field to retrieve any *name*-*value* pairs in the tag. These specify parameter settings for the earth view. In the **A** tag example above, the *projection* attribute specifies that the projection be mercator.

4.3.2 The Help Window

To open the Help window, click on the ***Help*** button in the main window.

The interactive portion of the Help window is on the left side of the upper pane. An image representing the application's main window is displayed next to some text; this illustrates VistaPoint's ability to display images next to text.

As you move the mouse cursor over the image, MS Windows like tips are displayed that explain the action associated to any sensitive text, and otherwise provide information on the underlying functionality.

The area covered by the image is subdivided in sensitive areas, using **AREA** tags in the HTML text. This implements an image map overlaid on the window image.

Each area definition includes an *href* field, in addition to the shape and coordinates defining the area. When the mouse pointer is moved over a sensitive rectangle, the widget invokes the **XnslNHtmlMouseOverCallback**. The application function attached to the callback pops a "tip" window similar to those used in MS Windows. The contents of the *href* field, received in the callback structure, is displayed in that window. For example, the area over Paris is defined by the following:

```
<area shape=rect href="Center the viewing position on Paris"
   coords="24,351,94,362" >
```

4.3.3 The Copyright Window

To open the copyright window, click on the ***Copyright*** button in the Help window.

This window is built with a VistaPoint widget that displays HTML text with one sensitive hyperlink, to close the window.

This window does not have a close button; it has a hyperlink with ***Close*** as its text. When you click the ***Close*** hypertext, the widget invokes the **XnslNHtmlActivateCallback.** The application function attached to the callback hides the copyright window.

4.4 YAB - Yet Another Browser

This application puts VistaPoint to work as a WEB browser. To evaluate how the widget performs with various HTML pages, launch *yab* and surf.

This application will also let you:

- analyze the HTML source of pages once they are loaded.
- display the tag tree.
- display the HTML source itself.
- list the tags of a given type and their attributes.
- trace loading actions as you surf.
- display the frame structure if any frames are loaded.
- display a graph of all URL's accessible from the page loaded.
- specify page parameters for WYSIWYG print preparation
- print to a PostScript file

To launch the application, first make sure that either the NSLHOME or the YAB-HOME environment variable is set properly:

NSLHOME, if defined, must be set to the NSL product installation directory.

If NSLHOME is not defined, YABHOME must be set to the directory where the yab demo was installed.

Change to the directory:

```
$NSLHOME/demos/vistapoint/yab
```

or to the directory:

```
$YABHOME
```

and launch the executable

```
./yab
```

You can specify one of the following command options:

- **-owncmap** - Make **yab** use its own colormap.
- **-urc** - Use RGBCube: colors are allocated in an rgb cube, allowing faster color allocation
- **-verbose -** Output connection and status messages in the xterm window during execution. Since the connection is made through the **wget** program, most of the output is from **wget**.

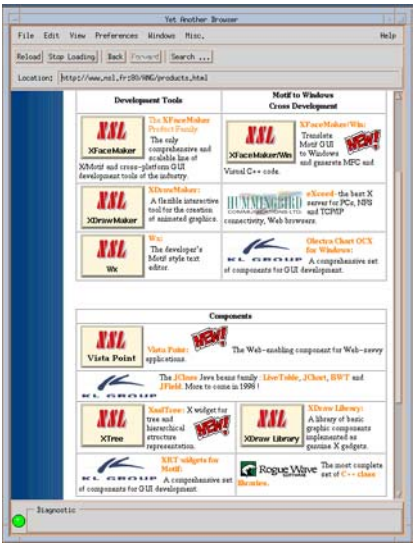
- **-help** - List command line options available and exit.

You can specify the URL that you want to load at start up by passing it as an argument on the command line:

```
./yab file:/usr/myhtmlfiles/myfirstfile.html
```

The default start up URL is `http://www.nsl.fr`

Your screen should look something like this, depending on the URL you load:



This application is built with XFaceMaker. The interface includes:

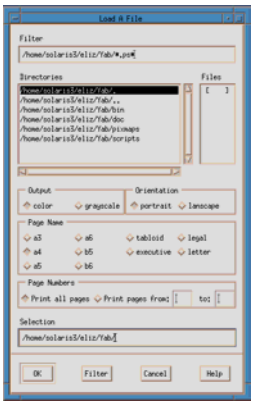
- a MenuBar.
- a set of command buttons.
- optional output fields where the current URL and other information can be displayed.
- the VistaPoint widget.
- a diagnostic field at the bottom of the window displays status information.
- a status icon that changes its color to red during page loading and is green when the widget is idle.

4.4.1 The MenuBar

The **File Menu** contains the following items:

- ***Load File ...*** Pops up the file selector to specify which file you want to load into the VistaPoint widget.
- ***Print ...*** Pops up a dialog box to let you
  - specify the file where you want to store the PostScript output of your page
  - specify the page orientation as landscape or portrait
  - specify the page size as “a3”, “letter”, etc...
  - specify the pages you want to output.

  The box looks like this:



  If you specify the same page name and orientation as in the Page Attributes box (see ***View Menu*** below) the page numbering, page breaks and image positions will be identical to those that were materialized on the display. This illustrates WYSIWYG printing with VistaPoint. The settings in this box determine the parameters passed to the *XnslHtmlSavePsWidget* function. They do not change any of the widget’s resource settings. Each page generated has a header and footer, that are specified by the yab application, through the callback structure, when the **XnslNhtmlPrintHeaderCallback** and **XnslNhtmlPrintFooterCallback** are invoked. Please refer to the source code to see how this is done.

- ***Exit*** to exit the application

The **Edit Menu** contains the following item:

- ***Search ...*** Pops up a Search Box dialog where you specify a search pattern and whether you want the search to be case insensitive or not. When you click on the

Search button in the box, the widget looks for a string that meets the search conditions in the page that is currently loaded. If a string is found in the page, it is highlighted and "Found" is written in the box's status field. Click on ***Next*** to find the next occurrence of the pattern. When no string is found, the status field displays "Not Found". A ***Close*** button lets you close the box when you no longer need it. Note that the same box is popped when you click on the ***Search ...*** command button below the MenuBar.

- ***Remove Image From Cache ...*** Pops a dialog box in which you specify the URL and size of the image that you want to remove from the image cache. Reminder: an image is stored in the cache according to its URL, its width and its height; it can be removed from the cache only if you specify the correct width and height, or zero if the IMG tag has no width/height information. You can use the ***Tag Info*** box (accessible through the Windows Menu) to find the IMG tags in the page. Click on ***Remove*** to remove an image that is not currently referenced (calls *XnslHtmlRemoveImageFormCache*). The status field to the right of the box displays *Removed* if the image was successfully removed from the cache and *Not Removed* otherwise. Click on ***Delete*** to remove an image as soon as it is not referenced (calls *XnslHtmlDeleteImageFromCache*). No status information is output in this case. Click on ***Close*** to close the box.

The **View Menu** contains the following toggles:

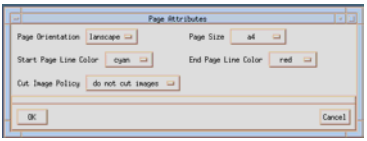
- ***Set Page Attributes...*** - Pops a dialog box in which you specify the page parameters that you want VistaPoint to use to render the HTML page. This illustrates the WYSIWYG capabilities of the widget for PostScript output preparation. If you specify *none* for the page name (**XnslNhtmlPageSize** resource), the HTML text uses the dimensions of the widget to determine the width and height of the page. No page breaks are shown. If you specify any other page name, the widget will materialize page breaks on the display according to the parameters you specify, once you click OK to validate your choices.It draws a beginning of page line, with the page number at the top of pages, and an end of page line, with the page number, at the bottom of pages.

  VistaPoint will generate a new page whenever it encounters a **<BRPAGE>** tag in the HTML text. This can be placed in the text, or within table cells: you control not only how text is broken up but also how tables span pages.

  Note the option menu concerning the Cut Image Policy - with this, you can specify how any image that spans two pages should be rendered: you can choose to cut the image wherever the page break occurs, to move any such image to the top of the next page, or to let floating images be cut while fixed images are moved to the top of the next page. This menu sets the **XnslNhtmlCutImagePol-**

icy resource.

The Page Attributes box looks like this:



The resources you this box sets, in addition to those mentioned above, are **XnslNhtmlPageOrientation**, **XnslNhtmlPageSizeStartColor**, and **XnslNhtmlPageSizeEndColor**.

- ***Show Location*** - If this toggle is set, the URL of the page currently loaded is displayed below the command buttons. This field is editable: you can use it to specify URL's of pages that you want to load, e.g.

  ```
  http://www.nsl.fr
  ```

- ***Show Current Directory*** - If this toggle is set, the value of the resource **XnslNhtmlCurrentDirectory** is displayed below the command buttons.
- ***Show Root*** - If this toggle is set, the value of the resource **XnslNhtmlRootUrl** is displayed below the command buttons.

The **Preferences Menu** contains the following items:

- ***Fonts...*** - Pops up a Font selection dialog that lets you specify the font with which the page is displayed. Specify a font family (both proportional and fixed), a font size, an increment. The font size specifies the base size in pixels of the font used to display the text (**XnslNhtmlFontSize** resource). The increment specifies the increment in pixels applied to the base font size when the FONT SIZE is changed by one unit in the HTML text (**XnslNhtmlFontSizeIncrement** resource). To specify the font size and the font increment, you can either enter a value, or use the scale slider, the scale being popped when you press mouse button 3 with the mouse cursor over the text field you want to change. Click ***Apply*** to apply your selection without closing the box, ***OK*** to apply your selection and close the box, ***Cancel*** to close the box without changing the font.
- ***Margins ...*** - Pops up a Margin selection dialog that lets you specify the margins with which the page is displayed. In addition to the margins, the box lets you specify the Line Spacing, i.e. the space, in pixels, between lines (**XnslNhtmlLineSpacing** resource), the Indentation, i.e. the number of pixels used to indent text (**XnslNhtmlIndentSpacing** resource), and the minimum cell spacing, i.e. the minimum cell padding in tables (**XnslNhtmlMinCellPadding** resource). To specify one of the numerical values, you can either enter the value, or use the

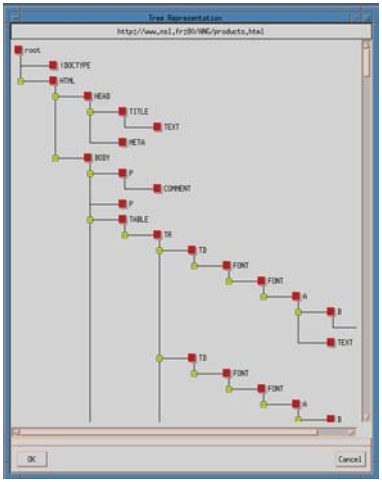
scale slider, the scale being popped when you press mouse button 3 with the mouse cursor over the text field you want to change. Click ***Apply*** to apply your selection without closing the box, ***OK*** to apply your selection and close the box, ***Cancel*** to close the box without changing the previous setting. The Margin and Spacing Box looks like this:

- ***Dither Images*** - If this toggle is set, images are dithered according to the Floyd Steinberg algorithm. This toggle controls the **XnslNhtmlDitherImage** resource.
- ***Auto Load Image*** - If this toggle is set (**XnslNhtmlLoadImagePolicy** resource set to XnslHTML_LOAD_IMAGE_FILE), images that are in local files are loaded automatically by the widget, while remote images are loaded under application control. If the toggle is not set (**XnslNhtmlLoadImagePolicy** resource set to XnslHTML_CALL_LOAD_IMAGE) all images are loaded under application control.
- ***Layout With Visible Width*** - If this toggle is set, the widget adjusts text layout to the width of the viewing window (**XnslNhtmlLayoutPolicy** is set to XnslHTML_USE_VISIBLE_WIDTH). If not, it adjusts text to the width of the widget (**XnslNhtmlLayoutPolicy** is set to XnslHTML_USE_WIDGET_WIDTH).
- ***Accept Frames*** - If this toggle is set (**XnslNhtmlAcceptFrames** resource set to True), any frames or framesets encountered in the page are loaded while, if the toggle is not set (**XnslNhtmlAcceptFrames** resource set to False), any frames and framesets in the page being loaded are ignored by the widget.

The **Windows Menu** contains the following items:

- ***Show Tree ...*** - If this toggle is set, a *Tree Representation* window is opened, in which the tree structure of the page currently loaded is displayed in an **XnslTree** widget.

The window looks something like this:

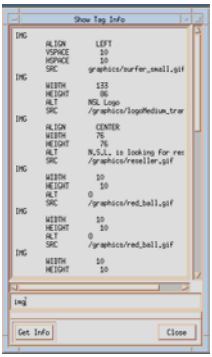


The items in the tree are the tags in the HTML page; their position in the tree reflects their position in the page hierarchy. Clicking on an item selects it; if the ***Source*** window is visible, when an item is selected in the tree, it is selected also in the source, the source window being scrolled as necessary to bring the tag within the view window. Pressing button 2 on an item in the tree pops a box that displays the contents of the tag. All sub-tree fragments in the structure have a +/- icon to show if the corresponding subtree is expanded or shrunk. To shrink a sub-tree, click on its - icon. To expand a sub-tree, click on its + icon.To scroll the window, use handmove (drag button 3) or the scroll bars. When, during parsing, the widget encounters missing tags, it inserts them so as to have correct HTML code. Any such tags are displayed in red in the tree. To close the *Tree Representation* window click on its ***OK*** or ***Cancel*** buttons in the window, or toggle the ***Show Tree*** item in the Windows Menu.

- ***Show Source ...*** - If this toggle is set, a window that contains the HTML text of the page is displayed. If the tree is also displayed and an item in the tree is selected, it is also selected in the Source window; the contents of the window is scrolled, if necessary, to bring the selection in the viewing window. To close the *Source* window, click on its ***OK*** or ***Cancel*** buttons, or toggle the ***Show Source*** item in the Windows Menu.
- ***Show History ...*** - If this toggle is set, a window that contains the history of sites visited, i.e. the titles of the pages loaded during a session is displayed in an **Xnsl-Tree** widget. Clicking on an item in the history tree reloads the corresponding

URL. To close the *History Box*, click on its ***OK*** or ***Cancel*** buttons, or toggle the ***Show History*** item in the Windows Menu.

- ***Show Tag Infos ...*** - If this toggle is set, a window is displayed that can list all the tags of a given type in the page currently loaded. It looks something like this:



  Enter the tag name, e.g. *img* in the input field at the bottom of the window, and click the ***Get Info*** button. All the tags of the type selected are displayed, along with any attributes. To close the box, click on its ***Close*** button, or toggle the ***Show Tag Infos*** item in the Windows Menu.

- ***Log Window ...*** - If the *Log Box* is open, the application outputs a trace of all actions in this window: which url it is accessing, the images it is loading, etc. Use this window to trace page loading as you surf.To close the box, click on its ***OK*** button or toggle the ***Log Window*** item in the Windows Menu.

- ***Show Graph ...*** - If the *Graph Representation* window is open, the links accessible from the page currently loaded are displayed in the window in an **XnslTree** widget. To close the window, click on its ***OK*** or ***Cancel*** buttons, or toggle the ***Show Graph*** item in the Windows Menu.

- ***Show Frame Info ...*** - If the *Frame Tree* window is open, and the page currently loaded contains frames (and the ***Accept Frames*** option is set) the page's frame structure is displayed in the window. To close the window, click on its ***OK*** or ***Cancel*** buttons, or toggle the ***Show Frame Info*** item in the Windows Menu.

The **Misc. Menu** contains the following items:

- ***Stop Image Animation*** - If this item is activated, the animation is stopped for all images currently loaded.

- ***Start Image Animation*** - If this item is activated, animation is restarted for all animated images in the page currently loaded.

4.4.2 Command Buttons

- ***Reload*** - Reload the current URL
- ***Stop Loading*** - Abort page loading
- ***Back*** - View the previous page
- ***Forward*** - View the next page
- ***Search ...*** - Pops a dialog where a regular expression to be looked for in the current page can be entered. Enter a search pattern and click the ***Search*** button to find the first occurrence of the string in the page. If the string specified is found, it is displayed in inverse video, and the page is scrolled, if necessary, to bring the string in the view window. Click the ***Next*** button to find the next occurrence of the string in the page. A toggle lets you specify whether the search should be case insensitive. Next to the toggle, a status field displays *Found* when the search yields a matching string, and *Not Found* otherwise.

4.4.3 Output Fields

- ***Location*** - This editable field displays the URL of the page loaded and lets the user enter the URL of a new page. The application attempts to load the URL specified when the user validates with Carriage Return. Use the **View Menu** to enable/disable the display of this field.
- ***Current Directory*** - This non-editable field displays the value of the **XnslNhtmlCurrentDirectory** resource that corresponds to the ***Location***. Use the **View Menu** to enable/disable the display of this field.
- ***Root*** - This non-editable field displays the value of the **XnslNhtmlRootUrl** resource that corresponds to the ***Location***. Use the **View Menu** to enable/disable the display of this field.

4.4.4 Diagnostic Field

The messages that are displayed in this field, at the very bottom of the window, let the user keep track of what is going on during page loading.

4.4.5 User Interaction

- ***Hyper links*** - As the user moves the mouse on the page, the cursor changes to the *hand* cursor when it is over a hyperlink. If the user presses mouse button 1 with the cursor over a hyperlink, the text changes its color to the value of the ALINK color (***Arm*** action). If the user drags the mouse away from the hyperlink, the text returns to the default color (***Mouse Move*** action). If the user releases button 1 on the hyperlink, the application fetches the associated URL (***Disarm*** action). Once

a hyperlink has been visited, its color is changed to the VLINK color.

- ***Scrolling*** - When the user presses mouse button 3, the cursor changes to the *fleur* cursor (***HandArm*** action). Dragging the mouse, with button 3 pressed scrolls the view window (***HandMove*** action). The cursor is changed to normal cursor when button 3 is released (***HandDisarm*** action).

CHAPTER 5: VistaPoint Functions

The functions that follow are in the VistaPoint (XnslHTML) library.

They are presented in the following format:

NAME

The C name of the function.

SYNOPSIS

The calling syntax, the arguments, and the return type.

DESCRIPTION

A brief description of the function.

EXAMPLE

When available, provides a short example illustrating how the function can be used. Another place to look for examples is in the source code of the demos. Ask your system administrator to install the demos if they are not available on your system.

SCOPE

States whether the function can be called from a FACE script only, from a FACE script and the application, or from the application only.

SEE ALSO

Lists related functions

NAME

XnslCreateHtmlWidget

SYNOPSIS

```
#include <Xnsl/Html.h>

Widget XnslCreateHtmlWidget
               (parent, name, arglist, argcount)
Widget              parent;
char                *name;
ArgList             arglist;
Cardinal            argcount;
```

DESCRIPTION

This is a convenience function to create a new VistaPoint widget, whose parent is *parent*. *arglist* specifies the argument list to override the resource defaults. *argcount* specifies the number of arguments in arglist.

SCOPE

Application, FACE

SEE ALSO

XtCreateWidget

NAME

XnslHtmlAbortRemoteImage

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlAbortRemoteImage (w, image_data)
Widget        w;
XtPointer     image_data;
```

DESCRIPTION

If, while images are being loaded under the application's control, the user takes an action to interrupt it, i.e. decides to do without images, the application has to inform the widget that it wants to interrupt the image loading process. Two situations arise:

- The **htmlLoadImageCallback** has not been completed: the application should set the *process_image* field of the callback structure to *False*. The **XnslHtmlAbortRemoteImage** function ***must not*** be called from the htmlLoadImageCallback.
- The application is still feeding image data to the widget, but outside the **htmlLoadImageCallback**: the application should call the **XnslHtmlAbortRemoteImage** function, passing it the *image_data* and the widget, *w,* received from the callback for that image.

Once image loading has been aborted, the *image_data* structure is freed by the widget. It can no longer be used.

If the application wants to load the same image later on, it has to start from scratch. The part of the image for which data had been provided is kept in the cache, with a "partial" attribute. The application will still have to provide all the image data for the image if and when it wants to load it.

It is not possible to interrupt image loading for images that are loaded entirely by the widget: images stored in local files are loaded at one go.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlDoRemoteImage

NAME

XnslHtmlAnchorOffset

SYNOPSIS

```
#include <Xnsl/Html.h>

int XnslHtmlAnchorOffset (w, anchor)
Widget                  w;
char                    *anchor;
```

DESCRIPTION

This function returns the vertical offset in pixels for the local anchor specified. The value returned is usually passed to the XnslHtmlScrollTo function.

If the anchor specified is not found, the function returns ***-1***.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlScrollTo

NAME

XnslHtmlComplete

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlComplete()
```

DESCRIPTION

XnslHtmlComplete terminates the use of the Html library. If the library contains license protection, calling its termination function releases its license token. Application programs may call this function safely when using the unprotected (run-time) version of the library.

SCOPE

Application, FACE

NAME

XnslHtmlComputeUrl

SYNOPSIS

```
#include <Xnsl/Html.h>

char*  XnslHtmlComputeUrl (w, root, url)
Widget              w;
char                *root;
char                *url;
```

DESCRIPTION

This function builds and returns a well formed URL using the parameters passed to it, as well as the resources **XnslNhtmlCurrentDirectory**, **XnslNhtmlRootUrl**, and **XnslNhtmlUrl** of widget *w*, as detailed below.

If the information provided does not lead to a well formed URL, the function returns ***0***. If it succeeds, it returns the ***URL*** in a string that should be freed by the application when it no longer needs it.

If a BASE tag exists in the page loaded, a root url and a current directory are deduced from the base. These supersede the resource settings for **XnslNhtmlRootUrl** and **XnslNhtmlCurrentDirectory** in the evaluation of the URL returned.

- if *url* starts with a # the function concatenates the *url* parameter to the **XnslNhtmlUrl** resource.
- if *url* is a well formed URL, the function returns a copy of the *url* parameter.
- if *root* is non-zero, the function concatenates *root* and *url;* if the result is a well formed URL, the function returns it.
- if the resource **XnslNhtmlCurrentDirectory** is non-zero and *url* does not begin with a *"/"* the function concatenates the content of the resource and *url*. If the result is a well formed URL, the function returns it.
- if the resource **XnslNhtmlRootUrl** is non-zero and *url* begins with a *"/"* the function concatenates the content of the resource and *url*. If the result is a well formed URL, the function returns it.
- if *url* begins with a *"/"* the function returns the string **file:** concatenated with *url*.

EXAMPLE

if **XnslNhtmlUrl** is `http://www.nsl.fr/index.html`
and *url* is `#product` the function returns:
`http://www.nsl.fr/index.html#product`

if **XnslNhtmlCurrentDirectory** is `http://www.nsl.fr/ANG/`
and *url* is `product.html`
the function returns:
`http://www.nsl.fr/ANG/product.html`

if **XnslNhtmlCurrentDirectory** is `http://www.nsl.fr/ANG/`
XnslNhtmlRootUrl is `http://www.nsl.fr/`
and *url* is `/graphics/logo.gif`
the function returns
`http://www.nsl.fr/graphics/logo.gif`

if **XnslNhtmlCurrentDirectory** is `http://www.nsl.fr/ANG/`
XnslNhtmlRootUrl is `http://www.nsl.fr/`
and *url* is `../graphics/logo.gif`
the function returns
`http://www.nsl.fr/graphics/logo.gif`

SCOPE

Application, FACE

NAME

XnslHtmlDeleteImageFromCache

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlDeleteImageFromCache
                        (w, name, width, height)
Widget                  w;
char                    *name;
int                     width
int                     height;
```

DESCRIPTION

This function will remove the image whose URL is *name,* whose size is the same as *height* and *width*, from the image cache when the image is no longer referenced. If the image is not referenced when this function is being executed, the image is removed from the cache immediately. If the image is referenced by at least one widget in the application, the image is marked so that it will be removed from the cache when its reference counter reaches zero.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlRemoveImageFromCache

NAME

XnslHtmlDoRemoteImage

SYNOPSIS

```
#include <Xnsl/Html.h>

int XnslHtmlDoRemoteImage (w, name, image_data,
                buffer, num_read, total_size)
Widget              w;
char                *name;
XtPointer           image_data;
char                *buffer;
int                 num_read;
int                 total_size;
```

DESCRIPTION

This function is used by the application to feed image data to the widget during remote image loading, or local image loading under application control. Image formats that can be handled by this function are **gif** and **jpeg**.

w is the VistaPoint widget ID

name is the name of the image being loaded

image_data is the opaque structure passed by the **htmlLoadImageCallback** in the XnslHtmlCallbackStruct. The application must pass this parameter "as is".

buffer is the content of the image. This is allocated by the application. Usually, the application will not have the entire content of the image the first time it calls this function. Successive calls to the function will pass the same buffer, enriched with new data as image data retrieval progresses.

num_read is the size of *buffer*, i.e. the number of characters of image content that the application is providing this time.

total_size is the total number of characters contained in the image.

The function returns:

-1 if the widget does not have enough image data to start drawing the image. The application must continue to provide data from within the **htmlLoadImageCallback**.

0 if the widget has enough data to start drawing the image but does not have

the entire image content. The application must continue to provide data. It can, however, do so outside the **htmlLoadImageCallback**.

1 if the full image content has been provided; *num_read* is equal to *total_size*. At this time, the widget frees the *image_data* structure which must no longer be used.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlAbortRemoteImage

NAME

XnslHtmlFlushDelayedDisplay

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlFlushDelayedDisplay (w)
Widget              w;
```

DESCRIPTION

The widget does not process repaint events immediately; it maintains a list of repaint rectangles and redisplays them all when the application returns to the top level loop.

If the application needs to redisplay the widget contents before returning to the main loop, it should call this function. The function explores the list of repaint rectangles and redisplays the required regions.

SCOPE

Application, FACE

NAME

XnslHtmlFreeFrameInfo

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlFreeFrameInfo (frame_struct)
XnslHtmlFrameInfo            *frame_struct;
```

DESCRIPTION

Free the tree of **XnslHtmlFrameInfo** structures returned by the *XnslHtmlGetFrameTarget* function.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlGetFrameTarget

NAME

XnslHtmlFreeTagInfoList

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlFreeTagInfoList(tag_info_list)
XnslHtmlTagInfo         *tag_info_list;
```

DESCRIPTION

Free the entire tag information list, *tag_info_list*.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlGetTagInfo, XnslHtmlSearchTagInfo

NAME

XnslHtmlGetFrameInfo

SYNOPSIS

```
#include <Xnsl/Html.h>

XnslHtmlFrameInfo * XnslHtmlGetFrameInfo (w)
Widget              w;
```

DESCRIPTION

This function lets the application retrieve the frame information of the page loaded in widget *w*. If the page currently loaded is not a framed page, the function returns ***0***. Otherwise, it returns a tree of **XnslHtmlFrameInfo** structures. Widget *w* should be the root widget, i.e. the widget that spawned the frame widgets.

The **XnslHtmlFrameInfo** structure is:

```
typedef struct _XnslHtmlFrameInfo {
char                          *name;
char                          *url;
Widget                        w;
struct _XnslHtmlFrameInfo     *children;
int                           num_children;
} XnslHtmlFrameInfo;
```

name is the name of the frame - this is used by the 'target' field of an anchor to reach the frame.

url is the URL loaded in the frame.

w is the ID of the frame or frameset VistaPoint widget.

children is a pointer to the **XnslHtmlFrameInfo** structures of any children (a frameset contains frames or framesets).

num_children is the number of children.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlFreeFrameInfo

NAME

XnslHtmlGetFrameTarget

SYNOPSIS

```
#include <Xnsl/Html.h>

Widget XnslHtmlGetFrameTarget (root, w, name)
Widget              root;
Widget              w;
char                *name;
```

DESCRIPTION

This function returns the widget ID of the frame specified as follows:

root is the ID of the widget from which frames have been spawned. This is usually the topmost widget but it can be one of the frame widgets in the hierarchy. It specifies the widget from which to start searching the hierarchy to find the frame specified.

w is the current frame widget, e.g. the widget that has invoked the activate callback. This parameter is useful only when *name* is _self or _parent. It is ignored otherwise.

name is the name of the target frame, or one of the special names _self, _parent, _top. If *name* is _self, the function returns *w*. If *name* is _parent, the function returns the widget ID of the widget that spawned *w*. If name is _top, the function returns the widget ID of the root widget, i.e. the widget that spawned the frames; this may be different from *root*. If a frame named name is found in the hierarchy, its ID is name is a valid frame name, the widget ID of the corresponding widget is returned. If no frame with name *name* is found in the widget tree starting at *root*, the function returns ***0***.

SCOPE

Application, FACE

NAME

XnslHtmlGetTagInfo

SYNOPSIS

```
#include <Xnsl/Html.h>

XnslHtmlTagInfo * XnslHtmlGetTagInfo (w, node)
Widget             w;
XnslHtmlNode       node;
```

DESCRIPTION

This function lets the application retrieve the attributes associated to **SCRIPT** or **APPLET** tags. The *node* argument that the application should pass is an opaque structure in which the widget has stored the *name value* pairs of the attributes associated to the tag. The **XnslHtmlNode** structure is returned in the node field of the **XnslHtmlCallbackStruct** by **theXnslNhtmlScriptAppletCallback**.

The **XnslHtmlTagInfo** structure in which the tag attributes are returned is:

```
typedef struct {
char                                *name;
unsigned long                       offset;
unsigned long                       len;
char                                *text;
XnslHtmlArg                         *args;
struct _XnslHtmlTagInfo             *next;
} XnslHtmlTagInfo;
```

Where *name* is the tag name, e.g. **SCRIPT,** *args* is a pointer on a list of *name value* pairs (see below) and *next* is NULL as, in this case, the list contains only one element.

The **XnslHtmlArg** structure in which the name value pairs are returned is:

```
typedef struct {
char                                *name;
char                                *value;
unsigned long                       offset;
unsigned long                       len;
struct _XnslHtmlArg                 *next;
} XnslHtmlArg;
```

Where *name* is the name of the entity whose value is returned in *value*. *next* is a pointer to the next *name value* pair (XnslHtmlArg structure), or NULL if this is the last *name value* pair for this tag.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlSearchTagInfo, XnslHtmlFreeTagInfoList

NAME

XnslHtmlLoadFile

SYNOPSIS

```
#include <Xnsl/Html.h>

int XnslHtmlLoadFile (w, url)
Widget              w;
char                *url;
```

DESCRIPTION

This function loads an HTML file into widget *w*. The file to load is specified in the string *url*.

The function returns ***0*** if the file was successfully loaded, and ***-1*** if an error occurred.

SCOPE

Application, FACE

NAME

XnslHtmlReForwardSearch

SYNOPSIS

```
#include <Xnsl/Html.h>

int  XnslHtmlReForwardSearch (w, from, pattern, ci)
Widget              w;
int                 from;
char                *pattern;
int                 ci;
```

DESCRIPTION

Find, in the HTML page loaded in widget *w*, the first occurrence of a string that matches regular expression *pattern*, starting from position *from*.

If *ci* is non-zero, the search is case insensitive.

The function returns the position of the last character of the matching string if one is found, or ***-1*** if no matching string is found. The matching string is displayed with inverse video.

EXAMPLE

```
pos = 0;
while (pos != -1){
     pos = XnslHtmlReForwardSearch
                          (w, pos, "nsl", 1);
}
```

SCOPE

Application, FACE

NAME

XnslHtmlRegisterConverters

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlRegisterConverters()
```

DESCRIPTION

This function adds the resource converters specific to the VistaPoint widget. The application should never need to call this function as the ClassInitialize method registers the converters.

SCOPE

Application, FACE

NAME

XnslHtmlRemoveImageFromCache

SYNOPSIS

```
#include <Xnsl/Html.h>

Boolean XnslHtmlRemoveImageFromCache
                          (w, name, width, height)
Widget                    w;
char                      *name;
int                       width
int                       height;
```

DESCRIPTION

This function removes the image whose URL is *name,* whose size is the same as *height* and *width*, from the image cache if the image is not referenced.

If the image is not referenced when this function is being executed, the image is removed from the cache immediately and the function returns *True*.

If the image is referenced by at least one widget in the application, the image is not removed from the cache and the function returns *False*.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlDeleteImageFromCache

NAME

XnslHtmlSavePsWidget

SYNOPSIS

```
#include <Xnsl/Html.h>

int XnslHtmlSavePsWidget
(w, file, page, orientation, grayscale, from, to)
Widget              w;
char                *file;
char                *page;
int                 orientation;
int                 grayscale;
int                 from;
int                 to;
```

DESCRIPTION

Output the HTML page currently loaded in widget *w* in PostScript format to file *file*, with page size *page* and orientation *orientation,* starting at page *from* and ending at page *to*.

The page name can be any of *"a3"*, *"a4"*, *"a5"*, *"a6"*, *"b4"*, *"b5"*, *"tabloid"*, *"executive", "legal"*, *"letter"*.

orientation can be **XnslHTML_PORTRAIT** or **XnslHTML_LANDSCAPE.**

grayscale can be 0 or 1. If 1, all text is black, background colors are ignored, images are rendered with gray scaling. If 0, the page is generated with the colors specified for text, background, images in the HTML page.

from specifies the first page number that is to be output; page numbering starts at one. *from* must be less or equal to the last page number.

to specifies the last page number that is to be output. If positive, it must be larger or equal to *from*, and at most equal to the last page. To output to the last page, specify *-1*.

The current value of the **XnslNhtmlCutImagePolicy** resource determines where page changes occur if any images straddle pages.

Page break tags in the HTML page are recognized (<BRPAGE>).

To obtain WYSIWYG PostScript output, i.e. the same page layout as on the

screen, make sure you specify the same page name and orientation as in the widget resources when calling this function.

Any background pixmap specifications are always ignored.

The function returns ***0*** in case of success and ***-1*** in case of error (unknown page size, can't open file, invalid *from* or *to* values, etc...).

The widget generates the postscript file from the HTML page rendered in the format specified. If the page is too wide to fit on the page size chosen, the portions that do not fit are ignored. Page breaks are handled automatically.

Header and footer text can be specified, through the callback structure, in the **XnslNhtmlPrintFooterCallback** and **XnslNhtmlPrintHeaderCallback**.

SCOPE

Application, FACE

SEE ALSO

XnslNhtmlPrintFooterCallback, XnslNhtmlPrintHeaderCallback

NAME

XnslHtmlScrollTo

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlScrollTo (w, x, y)
Widget                  w;
XnslHtmlPosition        x;
XnslHtmlPosition        y;
```

DESCRIPTION

This function scrolls the page loaded in widget *w* to the absolute positions *x* and *y*. The function scrolls so as to place the coordinate specified at the top left corner of the widget, or at the position closest to the top left corner that still leaves the viewing window full.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlAnchorOffset

NAME

XnslHtmlSearchTagInfo

SYNOPSIS

```
#include <Xnsl/Html.h>

XnslHtmlTagInfo * XnslHtmlSearchTagInfo(w, tag)
Widget             w;
char               *tag;
```

DESCRIPTION

This function lets the application retrieve the attributes associated to all the tags of a given type in the current page. The *tag* argument specifies the tag name (e.g. IMG) for which the application wants to retrieve the attributes. The function returns a linked list of all occurrences of the tag in the current page.

The **XnslHtmlTagInfo** structure in which the tag attributes are returned is:

```
typedef struct {
char                                *name;
unsigned long                       offset;
unsigned long                       len;
char                                *text;
XnslHtmlArg                         *args;
struct _XnslHtmlTagInfo             *next;
} XnslHtmlTagInfo;
```

Where *name* is the tag name, e.g. **IMG,** *args* is a pointer on a list of *name value* pairs for this occurrence of the tag (see below) and *next* is a pointer to the structure corresponding to the next occurrence of the tag, or NULL if this is the last one.

If the tag name specified is **TEXT**, the text contained in the text is returned in *text*.

The **XnslHtmlArg** structure in which the name value pairs are returned is:

```
typedef struct {
char                                *name;
char                                *value;
unsigned long                       offset;
unsigned long                       len;
```

```
struct _XnslHtmlArg                    *next;
} XnslHtmlArg;
```

Where *name* is the name of the entity whose value is returned in *value*. *next* is a pointer to the next *name value* pair (**XnslHtmlArg** structure), or NULL if this is the last *name value* pair for this tag.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlGetTagInfo, XnslHtmlFreeTagInfoList

NAME

XnslHtmlStartImageTimers

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlStartImageTimers (w)
Widget              w;
```

DESCRIPTION

Restart animation for all animated images in the page currently loaded in widget *w*.

When an animated image is first loaded, it is animated the number of times specified. When animation is restarted with this function, the images are again animated the number of times specified in the tag.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlStopImageTimers

NAME

XnslHtmlStopImageTimers

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlStopImageTimers (w)
Widget              w;
```

DESCRIPTION

Stop animation for all animated images currently loaded in widget *w*.

SCOPE

Application, FACE

SEE ALSO

XnslHtmlStartImageTimers

NAME

XnslHtmlWarning

SYNOPSIS

```
#include <Xnsl/Html.h>

void XnslHtmlWarning (w, format,...)
Widget              w;
char                *format;
.                           /* values to convert */
.
```

DESCRIPTION

This variable arguments function calls XtAppWarning to issue the message specified.

w is used to get the application context corresponding to the widget.

format is a printf-like format specification of the message. If *format* contains conversion characters (%d,%f,%s, etc.), the corresponding values are passed in the additional arguments.

The function builds the message from the *format* and any additional arguments and outputs it to the stderr stream.

SCOPE

Application, FACE

SEE ALSO

XtAppWarning (in the Xt Intrinsics Reference Manual)

Index